Memory and context engineering

Architecture 1: University teaching deck

How does an agent keep, retrieve, and manage the information it needs from one call to the next?

Memory types, short-term memory, RAG and agentic RAG, MemoryBank, A-MEM, Search-o1, and context engineering.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Memory foundations

Explain why an LLM is stateless and why memory is needed to turn a model into an agent. Distinguish working, episodic, semantic, procedural, and parametric memory.

Short-term memory

Describe how conversation history is stored and fed back, how the context window limits it, and how trimming and summarization keep it manageable.

Long-term memory and context

Trace RAG, MemoryBank, agentic RAG, A-MEM, and Search-o1. Explain context engineering: tracking, selecting, compressing, and ordering what the model sees.

Source: Ch. 4, PDF pp. 187–238

Without memory, an LLM starts from a blank slate

Stateless model

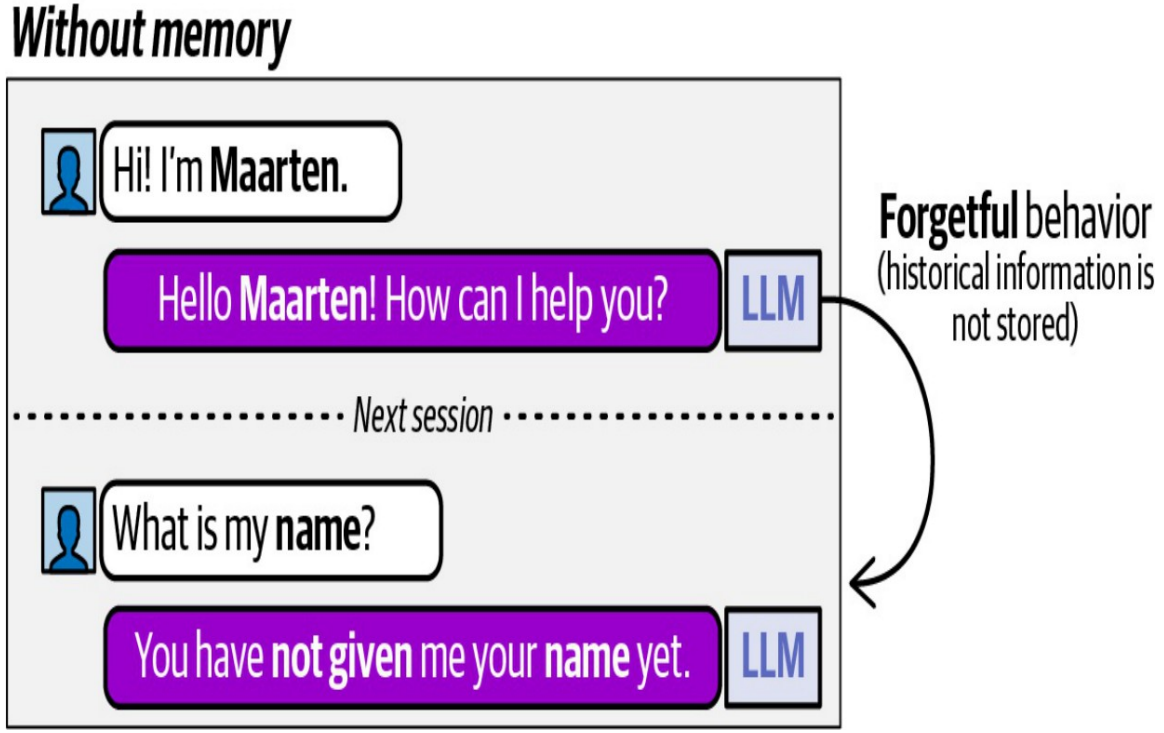
A language model on its own keeps nothing between calls. If it is told a name in one session, it cannot recall that name in the next because the information is not persisted.

Hosted assistants are augmented

Products such as ChatGPT and Claude are not plain LLMs. They are LLMs augmented with modules such as memory and tools, which is why they appear to remember.

Why it matters

Without memory, a personal assistant forgets past conversations, a coding agent cannot understand a whole codebase, and an agent may repeat an action it has already taken.



Grootendorst & Alammar (2026). Fig. 4-1, PDF p. 187

Memory covers more than the conversation

A broad definition

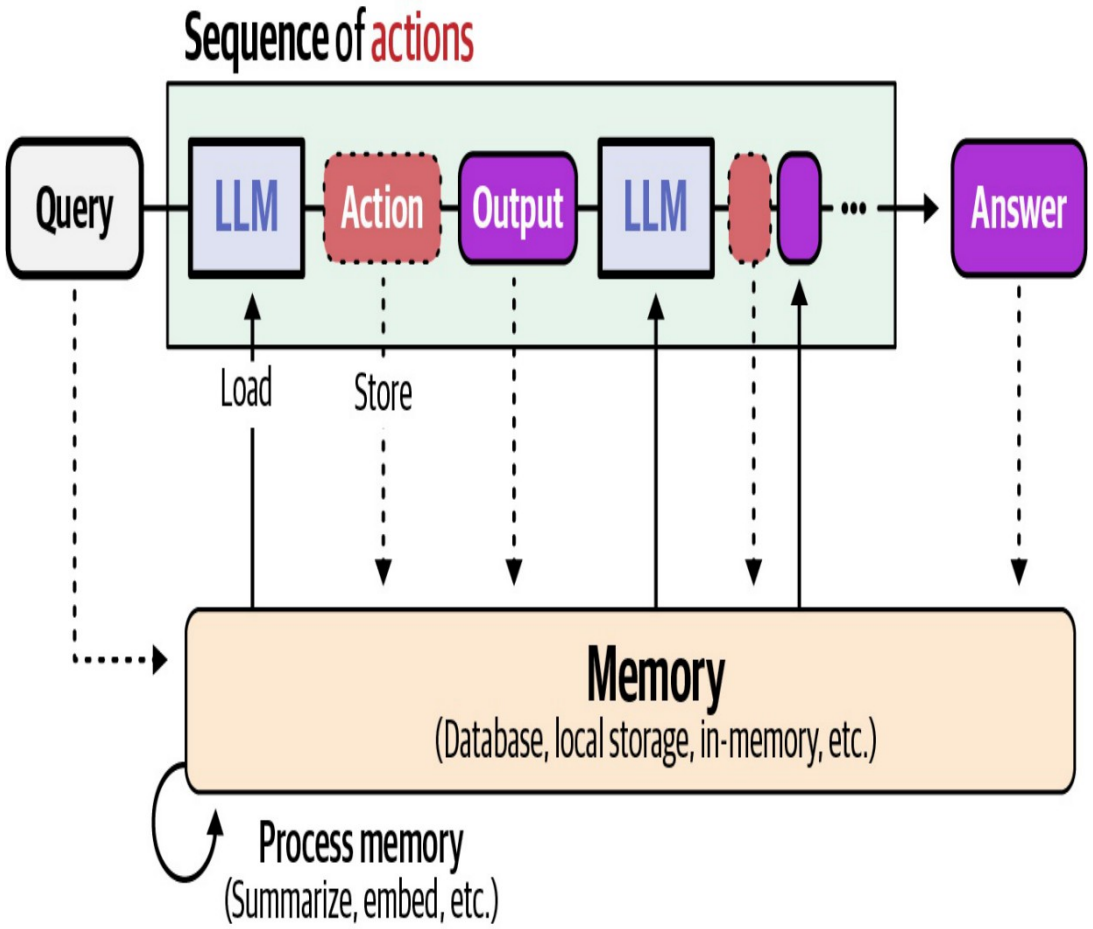
Memory includes the agent's past actions and also information beyond the agent-environment loop. For a coding agent, hosted documentation and issue pages count as memory too.

Store and use

Memory is not only remembering. New information must be stored, and choices are needed about what to keep, how to store it, what to retrieve, and what to forget or delete.

Learning from experience

Stored feedback and past errors help an agent handle similar tasks better. The underlying model can stay the same while behavior improves through accumulated experience.



Grootendorst & Alammar (2026). Fig. 4-2, PDF p. 188

Tools and planning depend on memory

The base of the agent

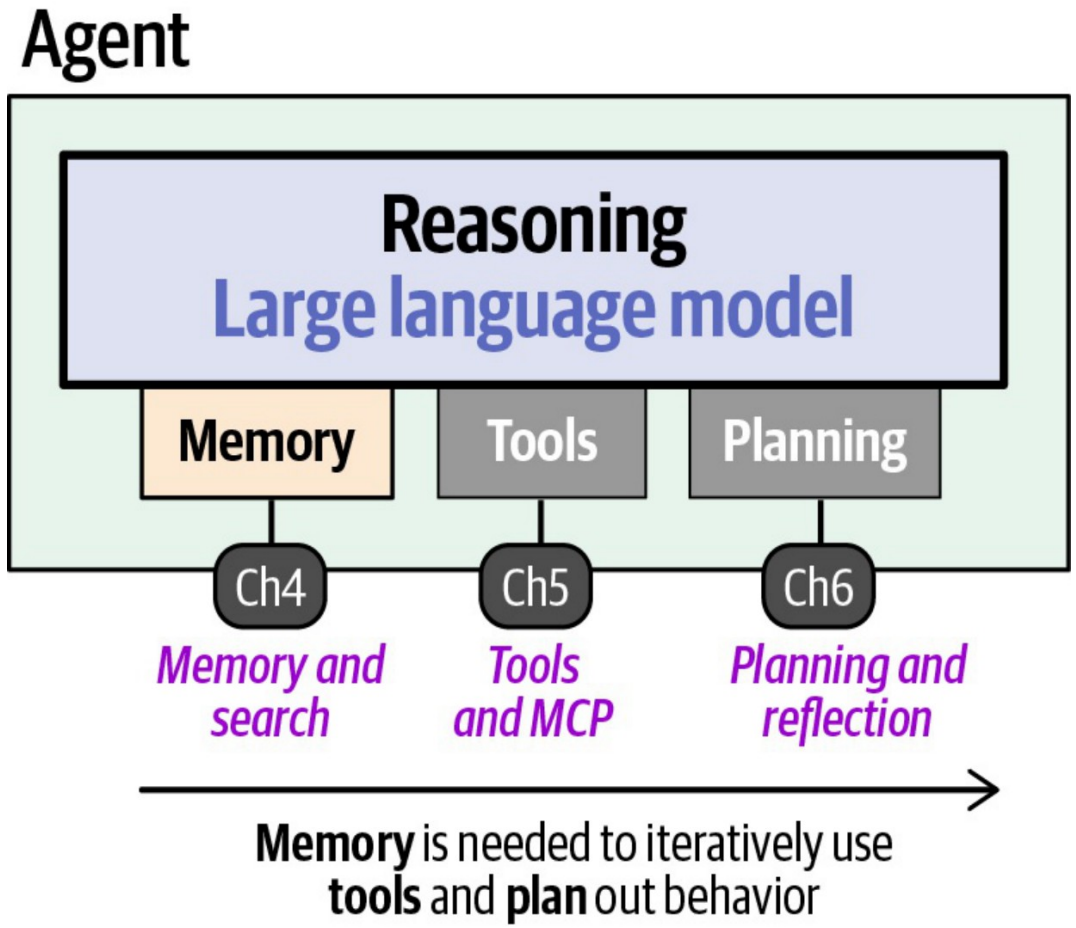
Memory is shown beneath tools (Chapter 5) and planning and reflection (Chapter 6). An agent cannot use tools or follow a plan if it keeps forgetting them.

Long traces

Tool use and planning produce long sequences of actions and results. Memory modules track this behavior across many steps.

Application-specific

What to remember, and how it is remembered, depends on the application. These design choices shape the agent's behavior.



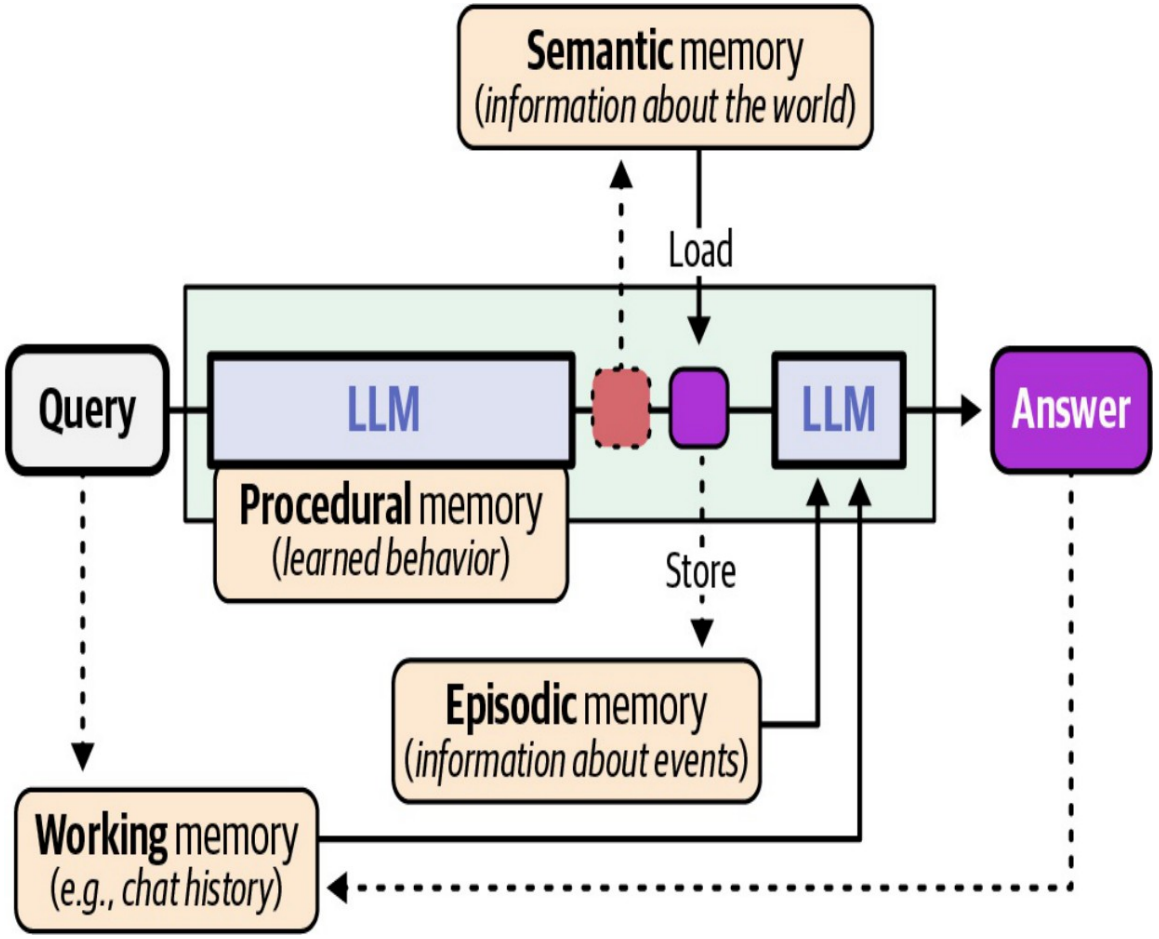
Grootendorst & Alammar (2026). Fig. 4-3, PDF p. 189

Four memory types organize an agent's memory

Working memory
Short-term, limited-capacity information held for decisions and reasoning. For an LLM it is the chat history that is fed back on every call.

Episodic and semantic
Episodic memory holds specific past events, such as actions taken and their outcomes. Semantic memory holds knowledge about the world, such as an external database or a codebase.

Procedural memory
Patterns for how to do things. In agents they appear in the system prompt, which persists across calls, and in the model's own parameters.



Grootendorst & Alammar (2026). Fig. 4-4, PDF p. 191

Memory types compared

Working memory

Human analogy: holding a phone number while dialing. Agent example: the running chat history. It lives in the prompt and is re-sent with each call.

Episodic memory

Human analogy: your last birthday party. Agent example: the specific actions taken so far and what resulted. It is typically stored as a log or database of past traces.

Semantic memory

Human analogy: knowing the capital of France. Agent example: Wikipedia, a company document store, or the project codebase. It is typically queried through retrieval.

Procedural and parametric

Human analogy: knowing how to write Python. Agent example: the system prompt and the knowledge stored in model weights. Recall from weights is useful but not a stable way to store specific facts.

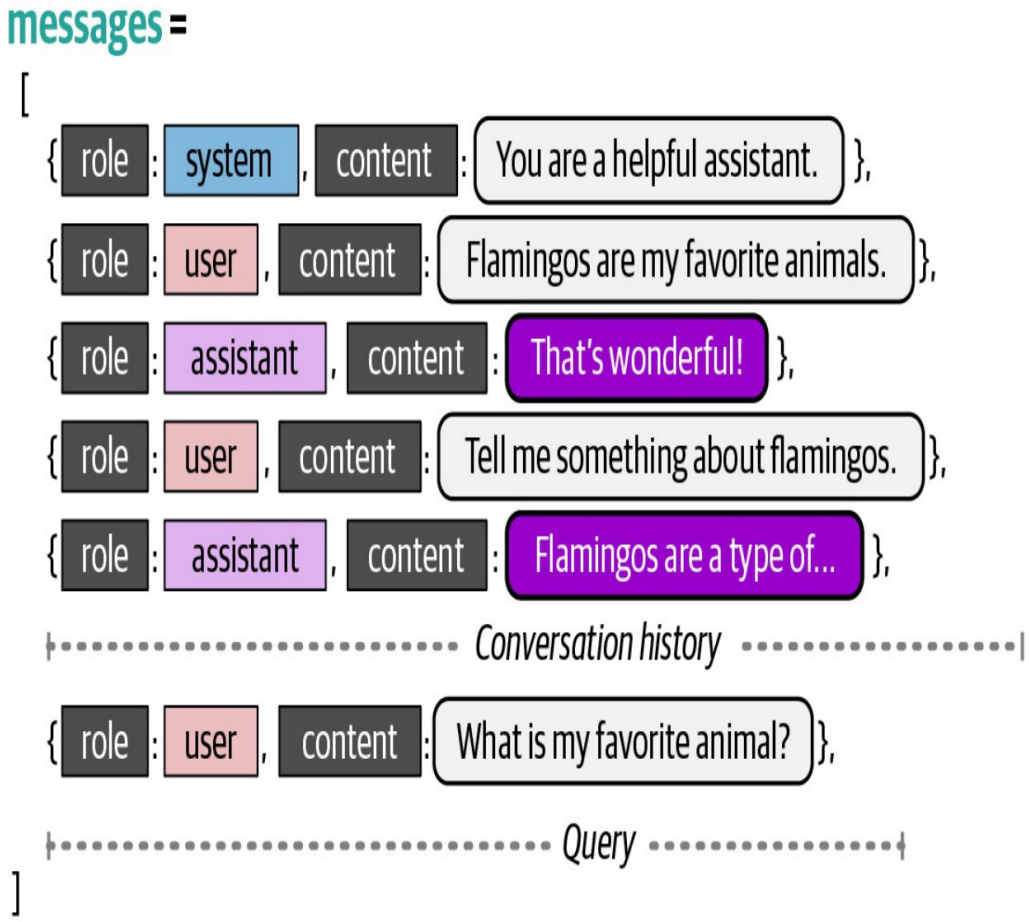
Teaching addition. Concepts: Ch. 4, PDF pp. 189–191

Conversation history is a list of messages

Message structure
Each message has a role (system, user, or assistant) and content. In the example, the user tells the model that they love flamingos.

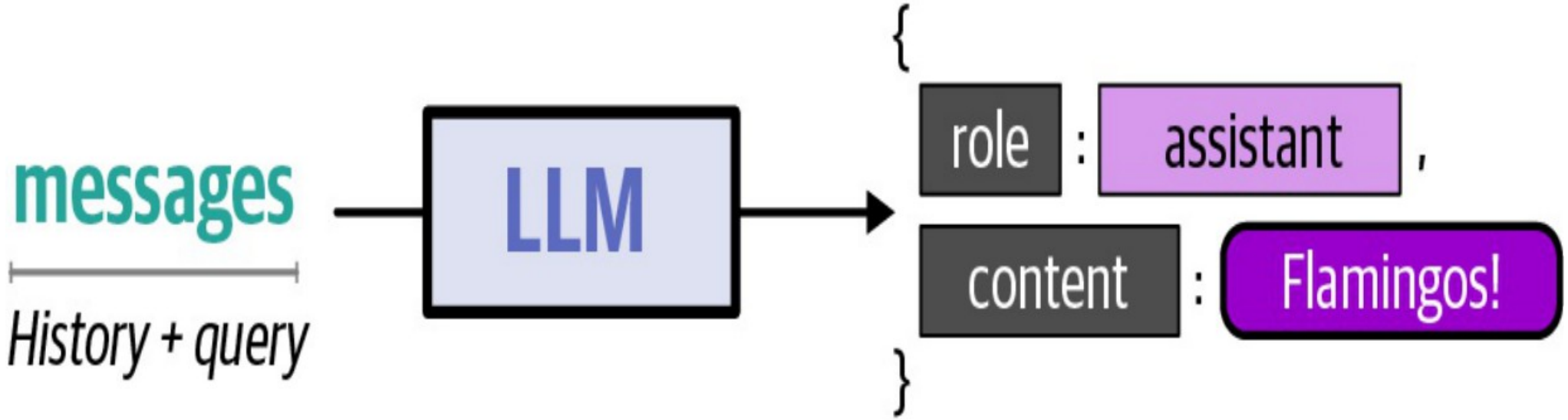
Updated every turn
The list is extended whenever the user or the assistant replies. It is then sent back to the model as input for the next query.

The query comes last
The new question is simply the final message. Any answer that depends on earlier messages can only come from the messages included in the same input.



Grootendorst & Alammar (2026). Fig. 4-5, PDF p. 193

The model is told the history; it does not recall it



Told, not remembered
The query does not make the model recall the past. The application supplies the whole history, which contains the fact that the user loves flamingos.

The Memory module
In the TinyAgent, Memory is a list of role-and-content dictionaries with two functions: one to add a message and one to get all messages.

Observed result
Without memory the agent answered that it did not know the authors' names. With memory it answered correctly, because the earlier turn was included in the prompt.

Grootendorst & Alammar (2026). Fig. 4-6, PDF p. 194

Adding Memory to the TinyAgent

Create and attach

A Memory object is created and passed to the agent. The agent starts with an empty list of messages and with empty slots reserved for tools (Chapter 5) and planning (Chapter 6).

Run and step

run adds the user's task to memory. The step function sends all stored messages to the LLM, adds the reply to memory, and records the reply in the trajectory.

Inspect the messages

Calling get_messages shows the exact input given to the model. Reading it is the quickest way to understand why an agent answered as it did.

Keep the raw record

The trajectory keeps the raw interaction even when memory is later trimmed or summarized. Later chapters add turns with only assistant roles, so user and assistant need not alternate.

Context length limits what a call can use

Context window

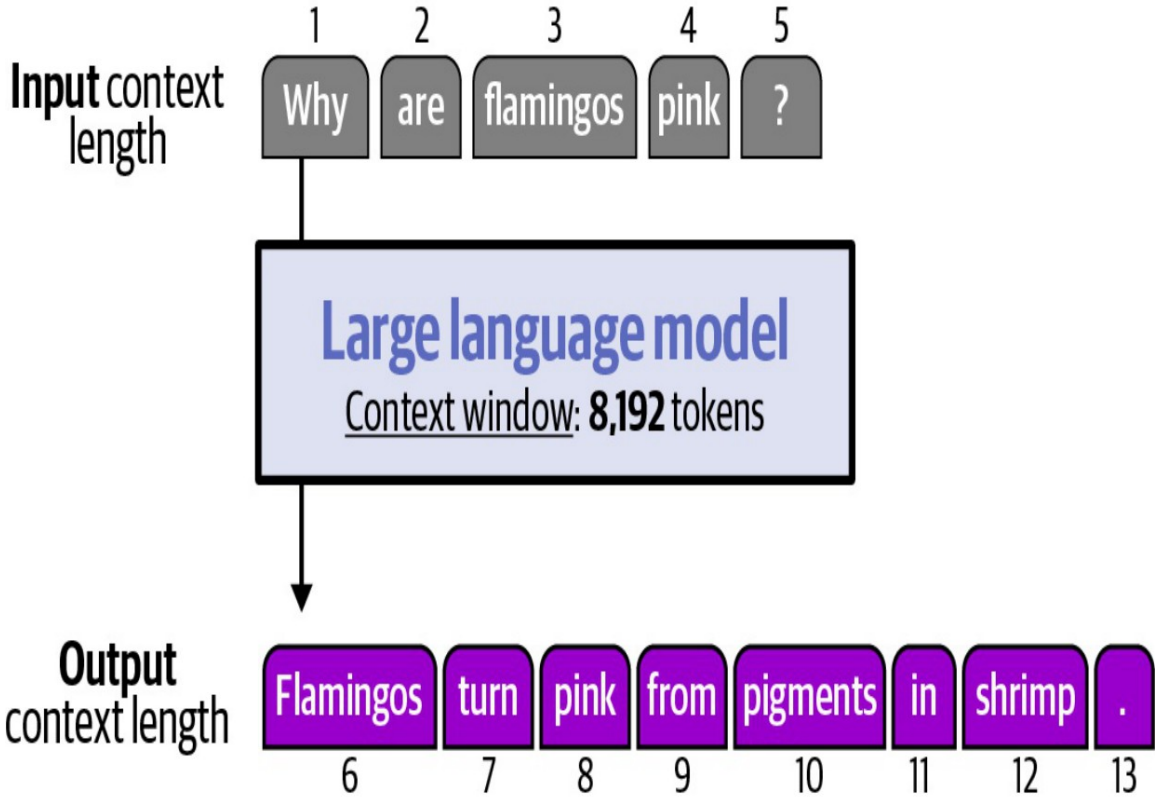
The window is the maximum number of tokens a model can process. Context length is the actual length of the input, the generated output, or both.

A small example

A short question and answer use only 13 tokens of a possible 8,192. Input tokens and output tokens count toward the same limit.

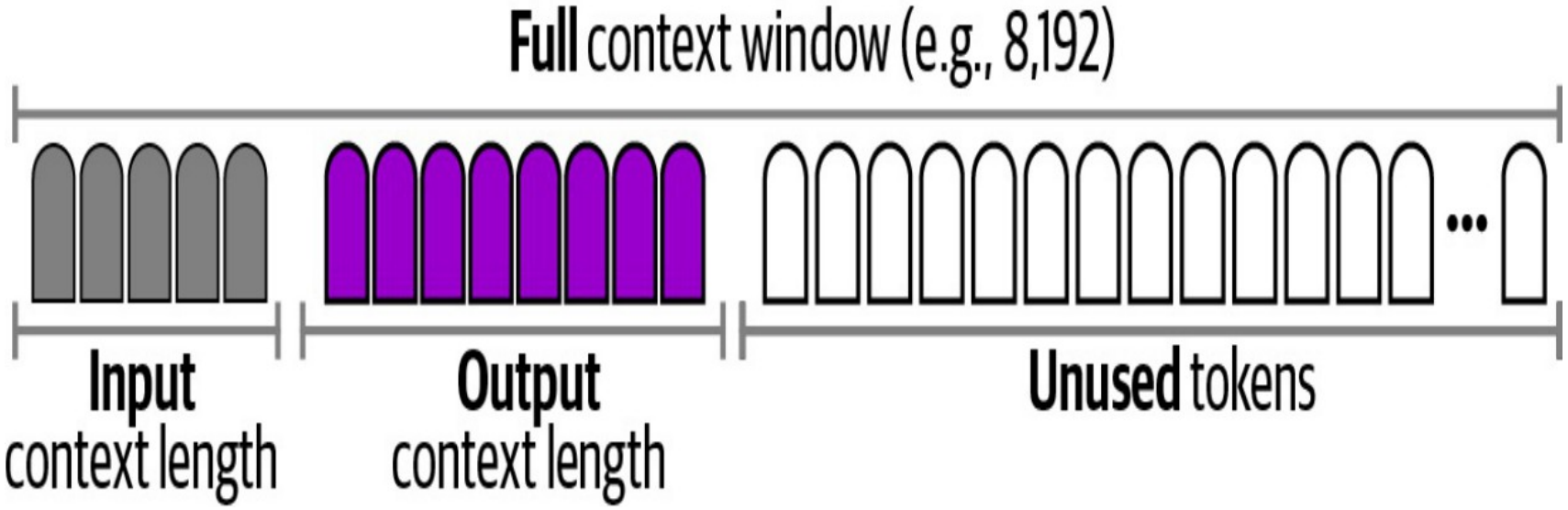
The growth problem

History grows with every turn and will eventually stop fitting. Agents therefore need ways to supply history without filling the window.



Grootendorst & Alammar (2026). Fig. 4-7, PDF p. 196

Input and output share one window



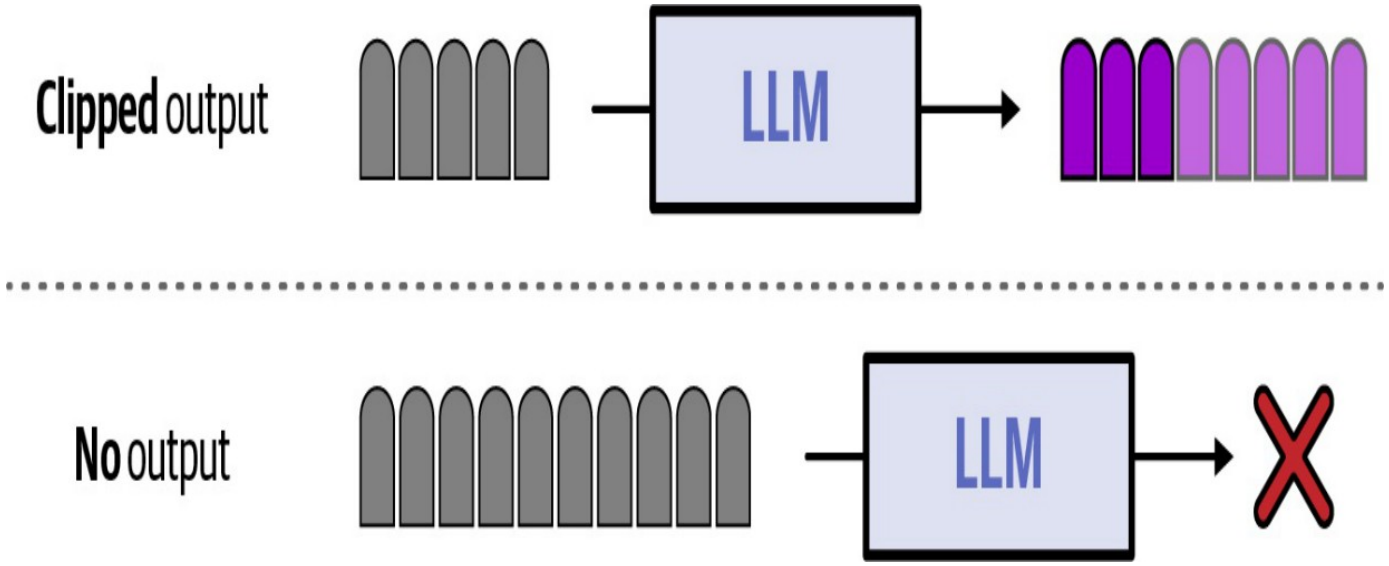
Three regions
The window holds input tokens, output tokens, and unused capacity. The unused space could hold extra information or reasoning tokens, as discussed in Chapter 3.

A shared budget
Every token of history, retrieved text, or instruction spends part of the same budget that the answer also needs.

Attention cost
Putting more information in the prompt also makes it harder for the model to attend to everything, so a longer history is not always a better one.

Grootendorst & Alammar (2026). Fig. 4-8, PDF p. 197

Overflow clips or blocks the response



Clipped output
If the history leaves little room, the generated answer may be cut short when it reaches the end of the window.

No output
If the input alone already exceeds the window, the model cannot process the prompt and cannot generate an answer.

Design response
Because of these failures, agents need a policy for shortening history. Two short-term policies follow, trimming and summarization. Long-term retrieval comes after them.

Grootendorst & Alammar (2026). Fig. 4-9, PDF p. 197

Trimming keeps only recent turns

The rule

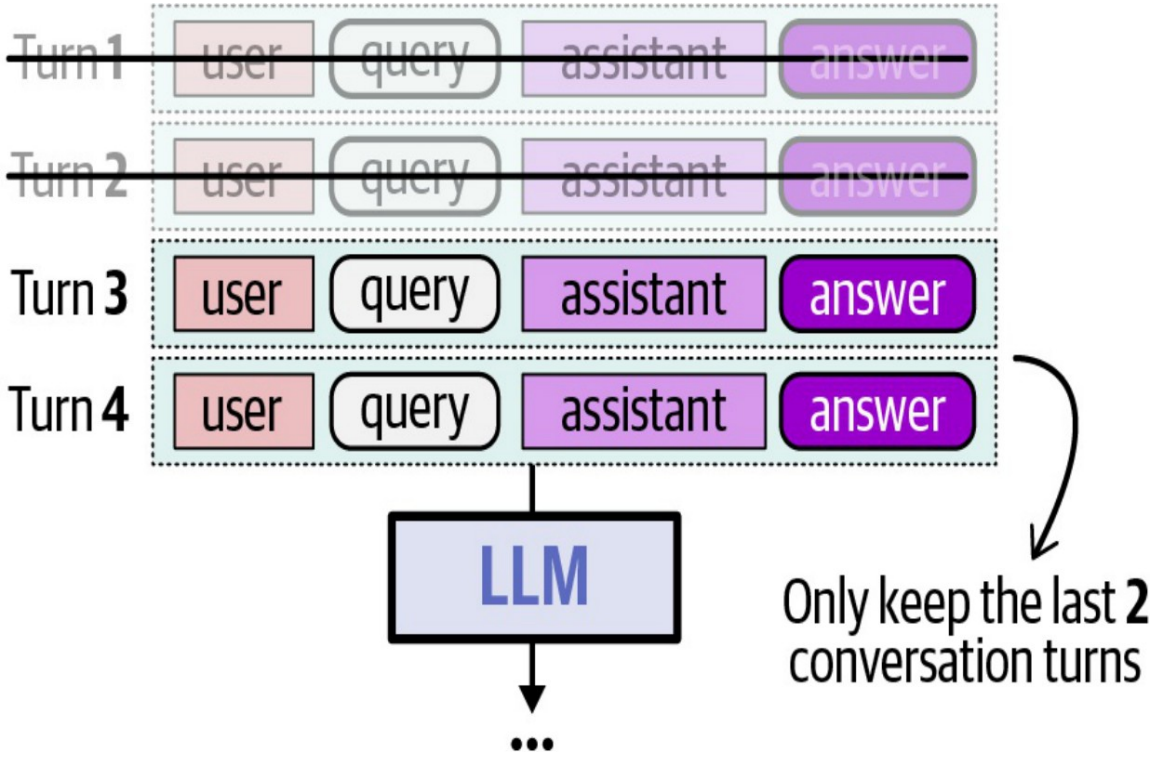
When the history grows too large, remove the oldest interactions until it fits. In the book's example the system message and the last two user and assistant turns are kept.

The cost

Trimming is simple and needs little code, but it can discard information that matters later, such as a name given at the start.

What persists

Information can still survive if the assistant keeps repeating it. The raw conversation should stay in the trajectory because memory itself may be stripped.



Grootendorst & Alammar (2026). Fig. 4-10, PDF p. 198

Summarization compresses past turns

A running summary

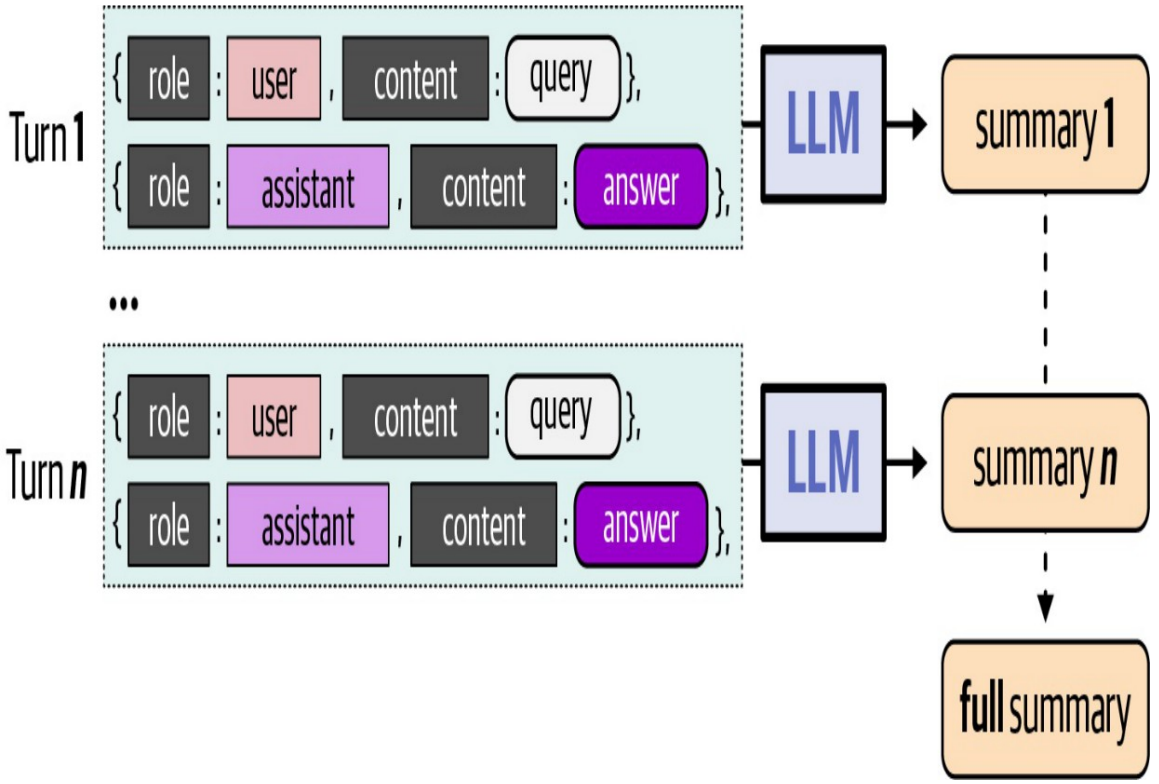
After each turn an LLM summarizes it and merges the result into a growing summary of the conversation. The same model or a different model can do the summarizing.

Slower growth

Summaries still accumulate, but much more slowly than raw history. Summarizing the summaries again saves more space but may discard important details.

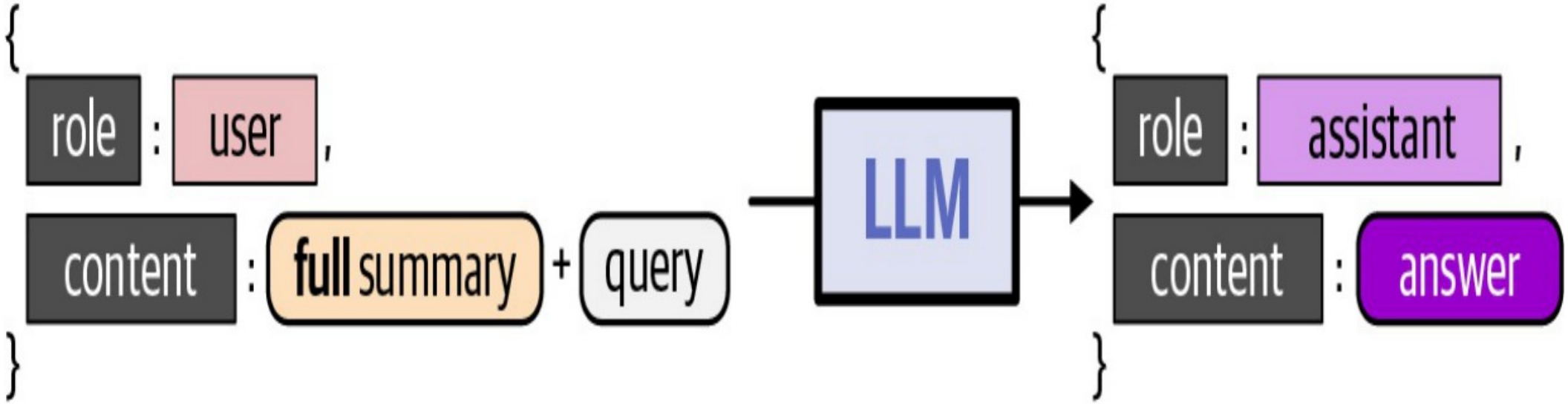
Variants

A system can summarize the last five turns at once, or keep one summary and ask the model to update it after each turn.



Grootendorst & Alammar (2026). Fig. 4-11, PDF p. 200

The summary replaces the raw history



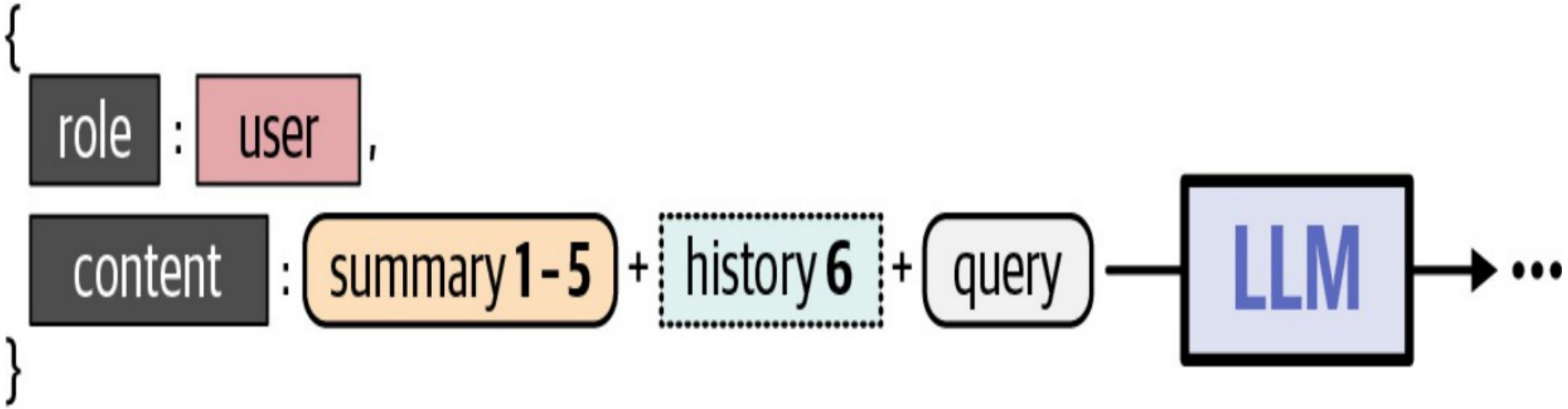
Input to the model
The model receives the summary together with the current query rather than every earlier message.

In the TinyAgent
A summarization memory stores the summary in a system message. After each completed turn it asks an LLM to update that summary.

Worked example
After a user named Sarah asked why flamingos are pink, memory held one system message summarizing the question and the answer about carotenoid pigments from their diet.

Grootendorst & Alammar (2026). Fig. 4-12, PDF p. 201

Partial summarization balances old and new



Mixed history
Older turns, for example turns 1 to 5, are summarized while the most recent turn stays as raw messages.

A balance
The summary carries compressed background. The raw recent turns keep exact detail, which is usually where precise wording matters most.

A judgment call
The right choice depends on what is important: the entire history, the recent history, or a summarized variant. Short conversations can often keep everything.

Grootendorst & Alammar (2026). Fig. 4-13, PDF p. 201

Short-term memory strategies compared

Keep everything

Strength: nothing is lost. Risk: the window fills and attention thins. Suitable for short conversations.

Trim

Strength: simple and predictable, with no extra model calls. Risk: early facts disappear. Suitable when only recent context matters.

Summarize all

Strength: keeps the gist of a long dialogue. Risk: extra model calls, and details can be compressed away or distorted.

Summarize old, keep recent

Strength: balances background with exact recent wording. Risk: more parts to design and test.

Teaching addition. Concepts: Ch. 4, PDF pp. 197–203

RAG ingestion builds long-term memory

Why long-term memory

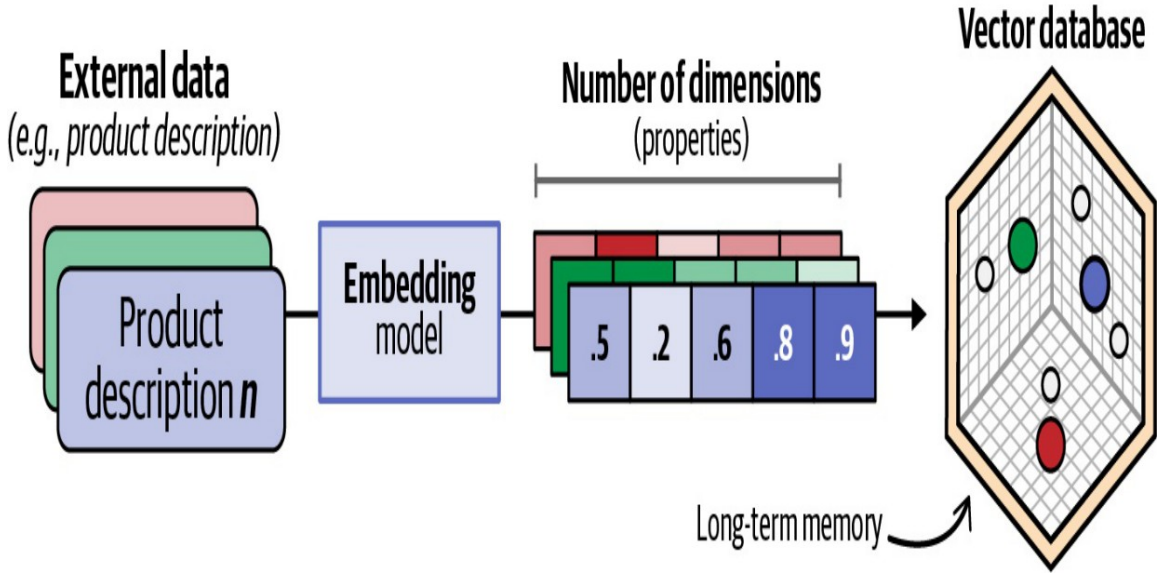
As history and actions grow, agents need external stores. Episodic memory holds past traces, while semantic memory holds knowledge such as an organization's documents.

Ingestion

External text is converted by an embedding model into numerical vectors that capture meaning. Texts with similar meaning receive similar vectors.

The vector database

The vectors are stored in a vector database. This database acts as the long-term memory that can later be queried.



Grootendorst & Alammar (2026). Fig. 4-14, PDF p. 204

RAG inference has four steps

Embed and retrieve

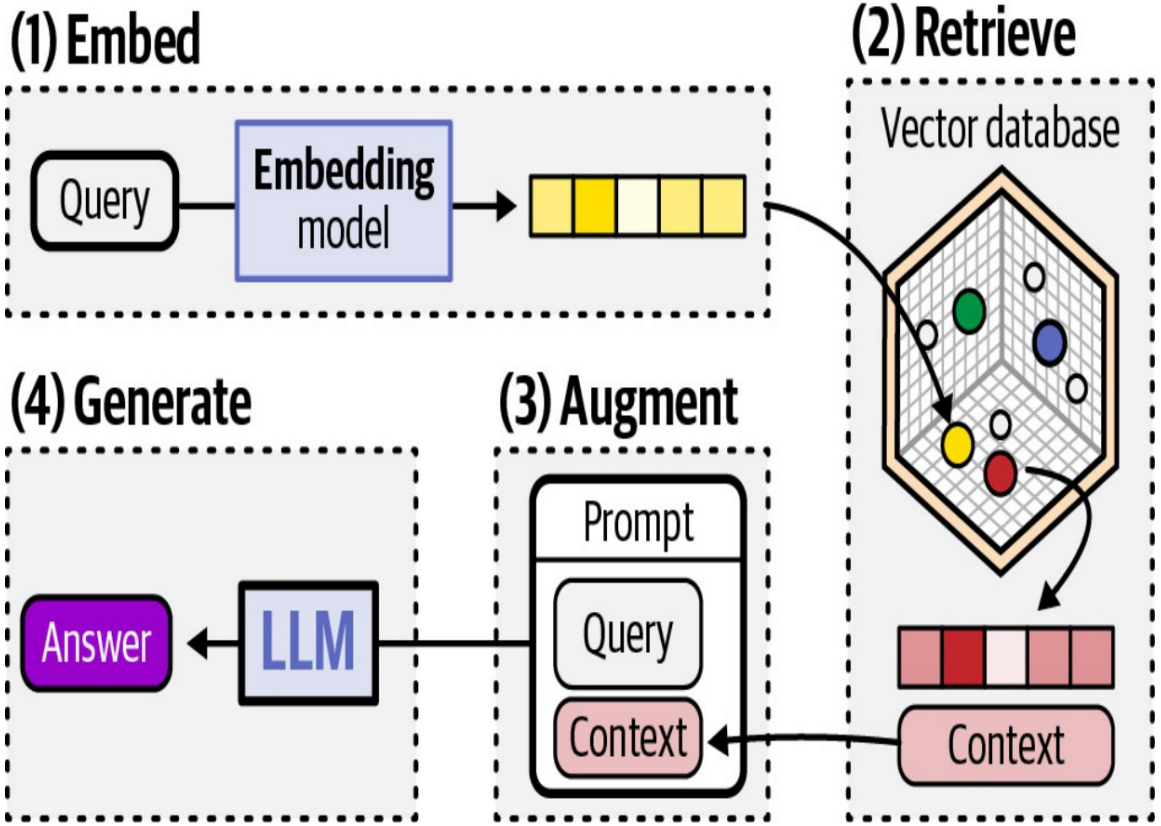
Step 1 embeds the user's query with the same model used for the documents. Step 2 compares it with the database and extracts the most relevant items.

Augment and generate

Step 3 adds the retrieved items and the query to the prompt. Step 4 uses this augmented prompt to generate the answer.

Purpose and limits

RAG is often used to reduce hallucination. It helps only if the retrieved context is relevant and correct. Relevance can also combine embeddings with keyword-style methods.



Grootendorst & Alammar (2026). Fig. 4-15, PDF p. 205

Similarity search inside RAGMemory

Embedding model

In the book's code a 308-million-parameter embedding model turns each text into 768 numbers, and it is reached through the same kind of interface as the LLM.

Cosine similarity

Similarity is measured by the angle between two vectors. In the example, I love flamingos scored about 0.64 against Flamingos are pink birds and about 0.35 against Dolphins use echolocation.

The five steps

Embed the documents, embed the query, compare them, return the highest-scoring documents, and add those to the prompt. The agent then correctly answered that Sarah's favorite animal is flamingos.

A limit to note

The module returned the top three documents whatever their scores, including unrelated ones. Accepting only documents above a minimum similarity is one possible refinement.

Grootendorst & Alammar (2026). Code and results, PDF pp. 205–209

MemoryBank adds forgetting and reinforcement

Forgetting curve

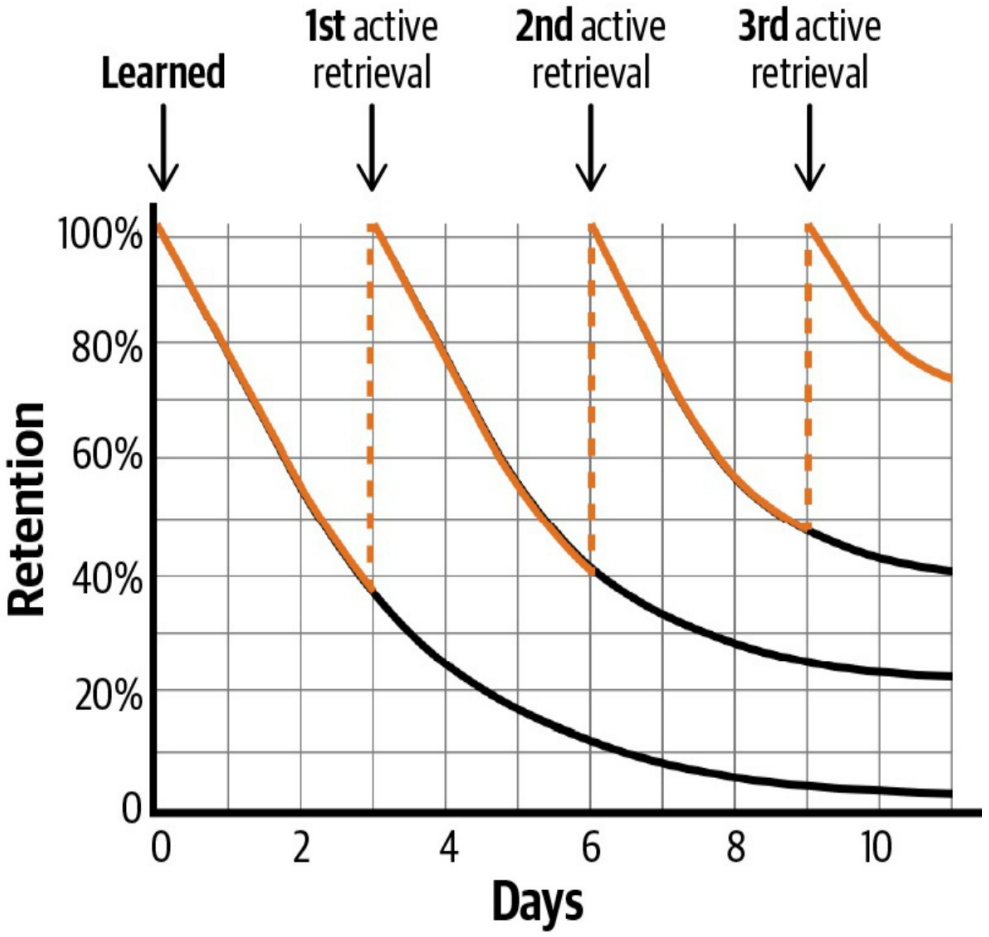
The Ebbinghaus curve shows knowledge decaying quickly, often drawn as exponential. Recalling information repeatedly, called spaced repetition, slows the decay.

Applied to LLMs

MemoryBank updates stored memories according to use. An item that is retrieved persists longer, while an item not retrieved for a while may be removed.

Beyond plain RAG

Standard RAG stores and retrieves. MemoryBank continuously updates its store so that memory is selectively preserved or forgotten.



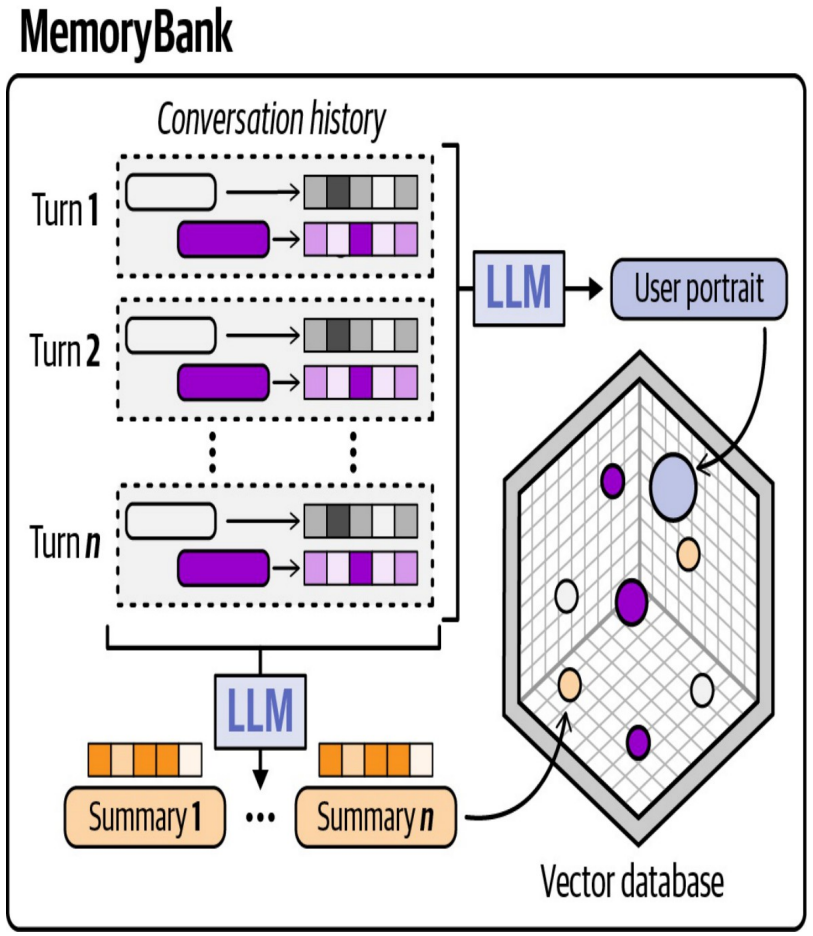
Grootendorst & Alammar (2026). Fig. 4-16, PDF p. 210

MemoryBank keeps three kinds of memory

Conversation history
Raw multi-turn conversations form the base record of what was said.

Summaries of past events
An LLM generates summaries from the conversation history. Summaries and turns are embedded so they can be retrieved.

User portrait
The LLM summarizes the user's personality traits and emotions. The portrait is updated dynamically and always passed as additional context.



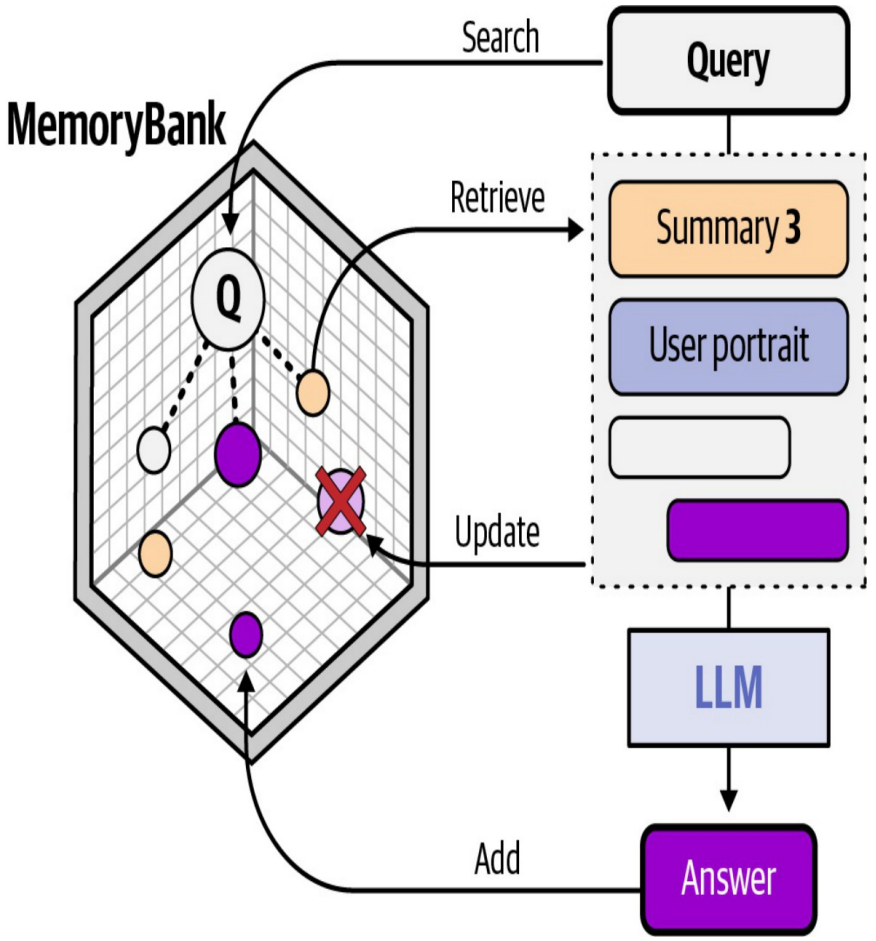
Grootendorst & Alammar (2026). Fig. 4-17, PDF p. 212

MemoryBank retrieval strengthens used memories

Query and retrieve
The query is embedded. Related conversation turns and summaries are retrieved together with the user portrait.

Update strength
When conversation turns are retrieved, their strength is updated, which makes them less likely to be removed from the MemoryBank.

Generate and add
The retrieved context and the query are the input for the answer, and the figure shows the new exchange being added back to the store.



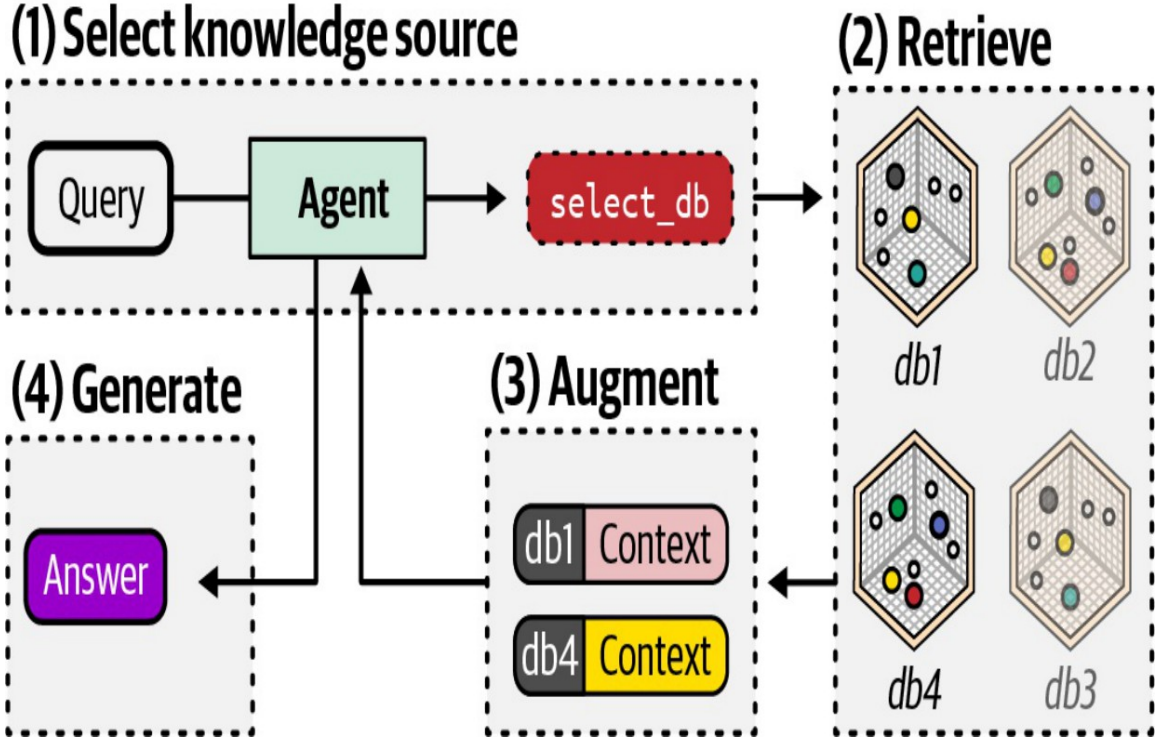
Grootendorst & Alammar (2026). Fig. 4-18, PDF p. 213

Agentic RAG gives the agent control

Vanilla RAG
The vector database is the long-term memory, but the LLM receives only what a fixed retrieval step supplies. It has no say over what is retrieved.

Agentic RAG
An agent can use the database as a tool, decide which source to query, and then decide whether to answer or retrieve again.

A router
In a single-agent system the agent acts as a router over several knowledge sources, treating each as a tool rather than a static step before generation.



Grootendorst & Alammar (2026). Fig. 4-19, PDF p. 214

One agent can chain several retrievals

Dynamic decisions

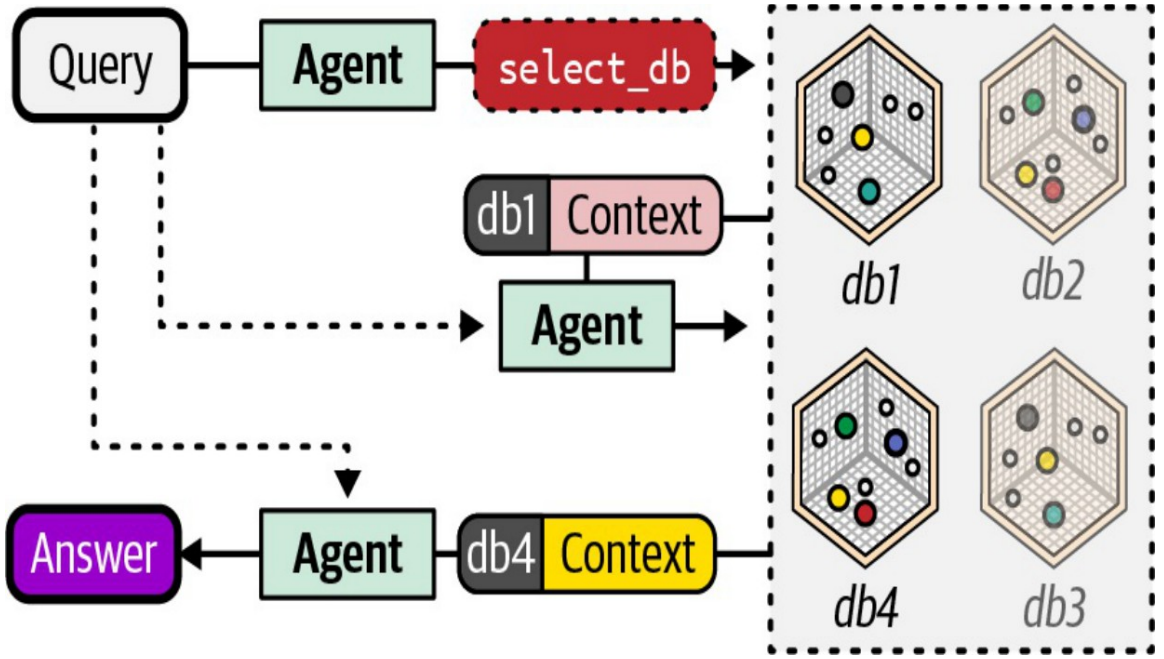
The agent can retrieve from one source and then use what it found to choose a follow-up search in another database.

Not only the query

Retrieval does not have to depend only on the original question. Intermediate results can steer later searches.

Agency

The book says LLM for RAG and agent for agentic RAG to signal that the agent decides how many times to query semantic memory until it has enough context.



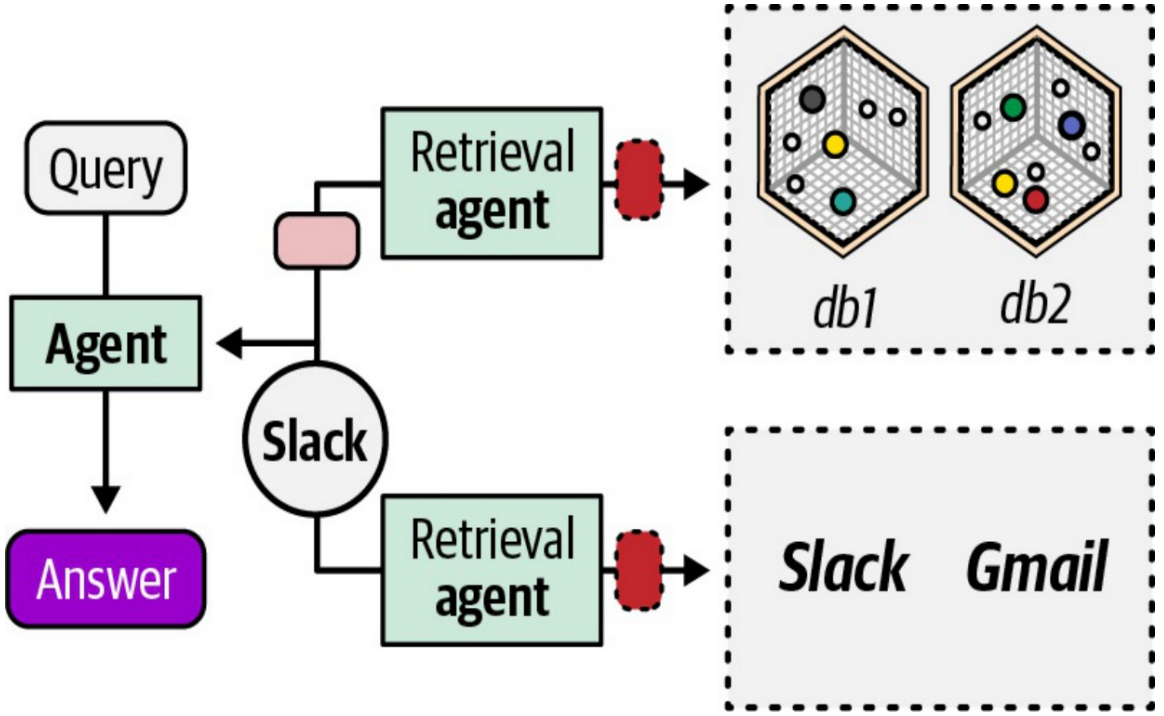
Grootendorst & Alammar (2026). Fig. 4-20, PDF p. 215

Multi-agent retrieval adds specialists

Coordinated specialists
Smaller retrieval agents, each specialized in one source, are coordinated by a single agent with more capabilities.

Not only vector databases
Sources can also include web search or APIs, such as Slack or Gmail, as the figure shows.

Start simple
The authors advise starting with a single-agent system as a baseline to minimize complexity before adding more agents.



Grootendorst & Alammar (2026). Fig. 4-21, PDF p. 216

Single-agent versus multi-agent RAG

Single-agent: advantages

Cost effective, because fewer API calls are needed. Simpler to debug, because there are fewer dependencies.

Single-agent: disadvantages

A single point of failure, since errors may compound without external feedback. A lower accuracy ceiling when the context is overloaded with sources.

Multi-agent: advantages

Modular, since specialized LLMs can be replaced easily. A higher accuracy ceiling, because agents can give each other feedback and check for hallucinations.

Multi-agent: disadvantages

Higher cost from running several agents in parallel. More complexity, which makes it harder to locate errors and relate them to the whole flow.

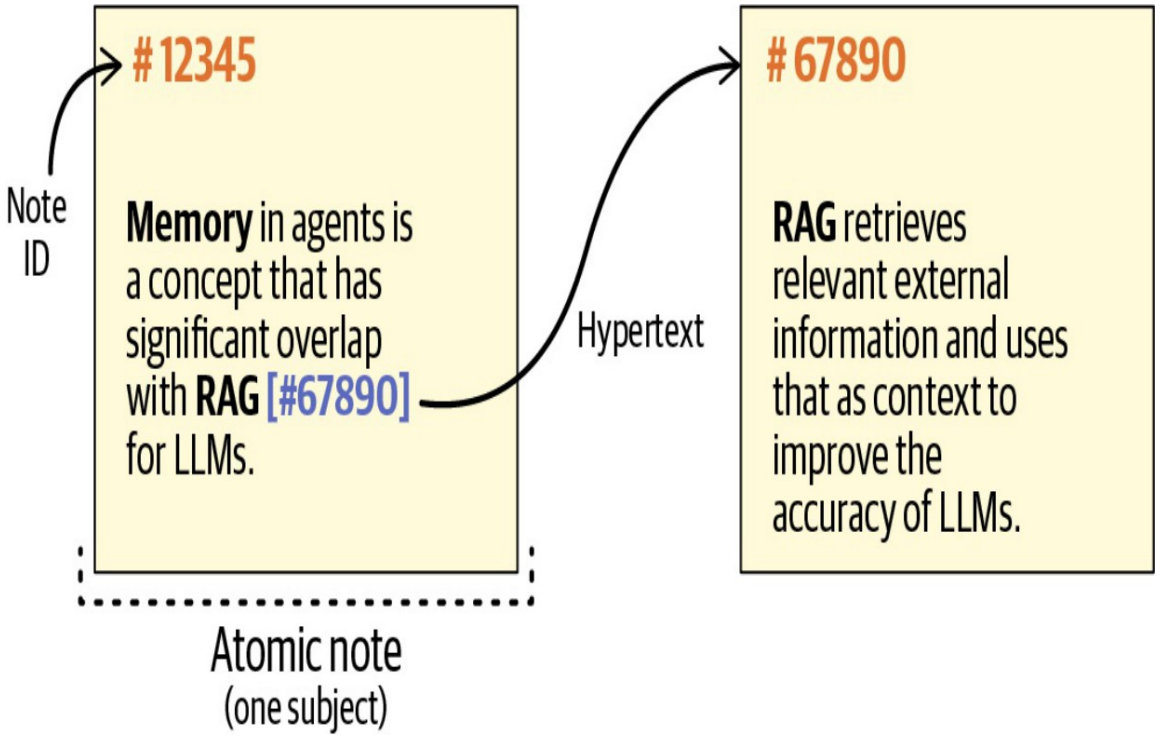
Grootendorst & Alammar (2026). Table 4-1, PDF pp. 216–217

A-MEM borrows from Zettelkasten note-taking

Atomicity
Each note holds one unit of knowledge, one atom, such as a short description of how memory works in agents.

Hypertextual notes
Notes refer to and expand on each other. A note about agent memory can link to a note about RAG.

Personalization
Interlinked notes form a web of ideas that reflects the individual's own sets of ideas.



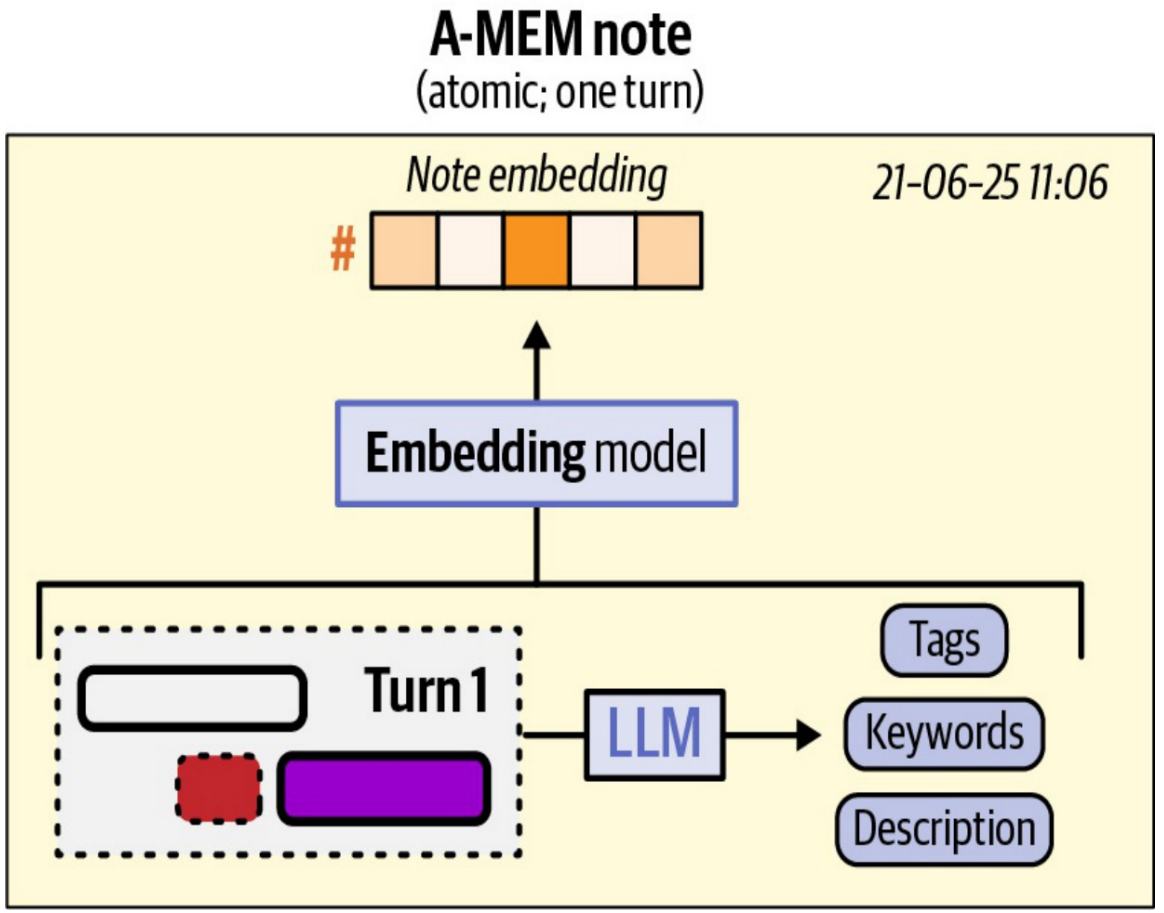
Grootendorst & Alammar (2026). Fig. 4-22, PDF p. 218

An A-MEM note stores one interaction

Note contents
The original interaction (one turn), its timestamp, and LLM-generated keywords, tags, and a contextual description.

One embedding
All fields except the timestamp are concatenated and embedded as one vector for the note. The timestamp is kept as metadata that can be queried.

Atomic by design
Because each note covers a single interaction, A-MEM follows the Zettelkasten principle of atomicity.



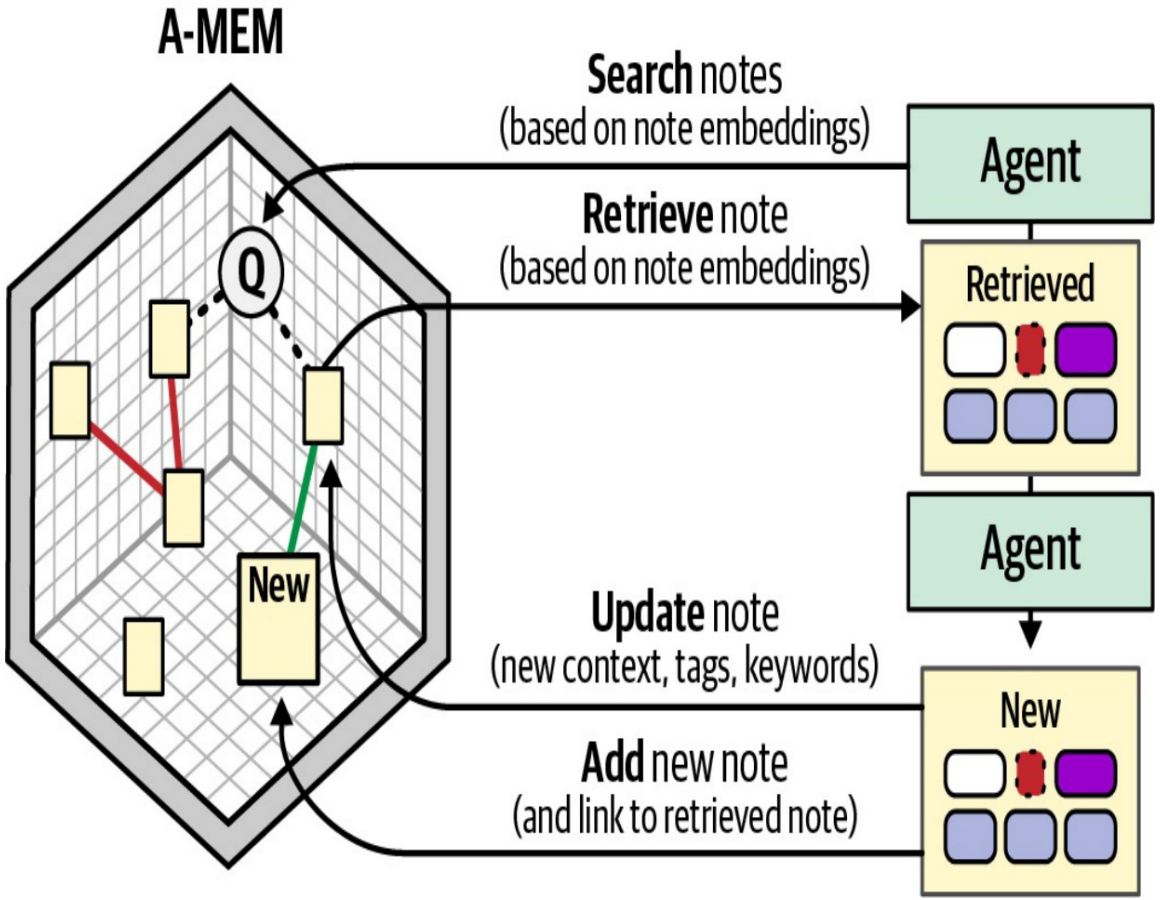
Grootendorst & Alammar (2026). Fig. 4-23, PDF p. 219

A-MEM links and updates memories

Linking
The new note's embedding drives a similarity search. The top-K similar memories become candidates, and an LLM decides which of them to link.

Evolution
After linking, the LLM updates tags, keywords, and descriptions in light of the new memory. Older notes change as newer ones arrive.

Agent access
The agent can search for notes related to a query and may also retrieve all notes linked to a retrieved note.



Grootendorst & Alammar (2026). Fig. 4-24, PDF p. 220

Search-o1 retrieves during reasoning

Search inside thinking

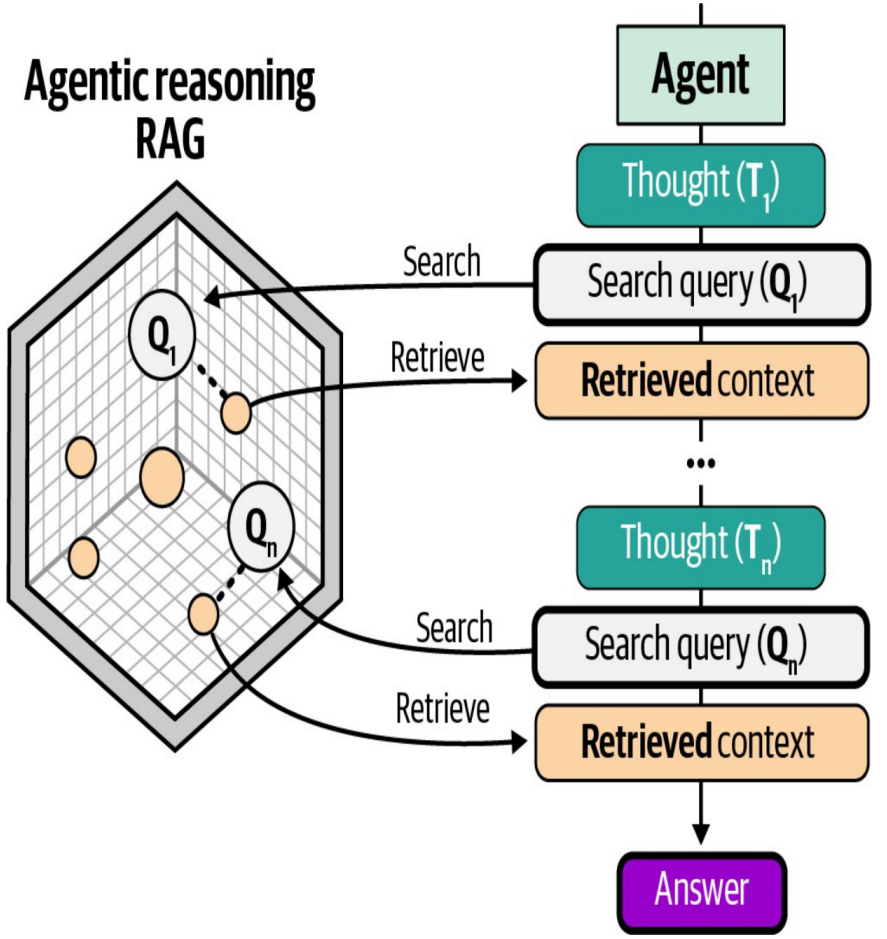
Instead of supplying documents before the model starts, retrieval happens during the reasoning process, so the model can search when it finds a gap.

Special markers

The model wraps a search in begin and end search-query tokens, and retrieved text is wrapped in begin and end search-result tokens.

Iterative refinement

Reasoning can pause, search, and continue until the model is confident, all within one call rather than across repeated calls.



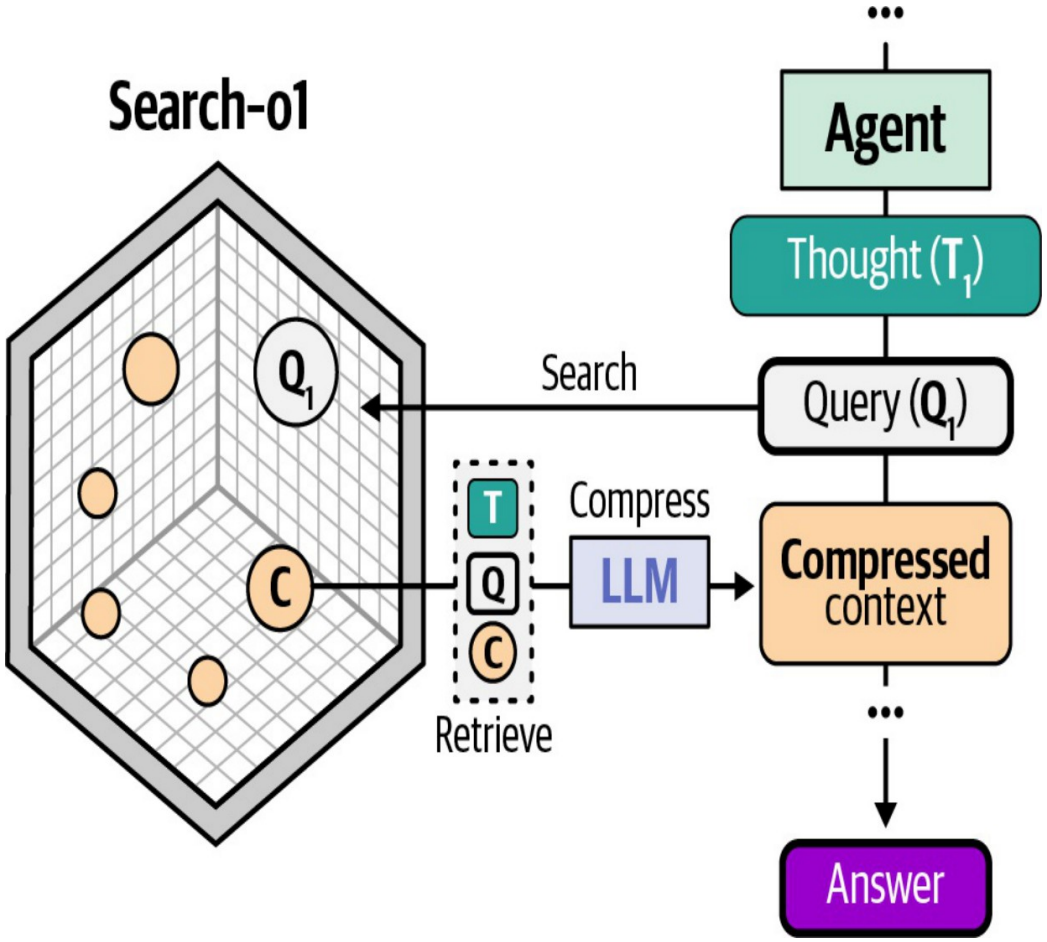
Grootendorst & Alammar (2026). Fig. 4-25, PDF p. 221

Search-o1 condenses what it retrieves

The problem
Retrieved documents are often long and contain irrelevant material that can disrupt the flow of reasoning.

Reason-in-Documents
A module takes the search query, the retrieved documents, and the reasoning trace, and condenses them into focused reasoning steps.

Worked example
For why flamingos are pink, a first search finds that dietary pigments are responsible. A later search in another source clarifies that they are carotenoids found in brine shrimp.



Grootendorst & Alammar (2026). Fig. 4-26, PDF p. 222

Context is more than the user's prompt

What goes in

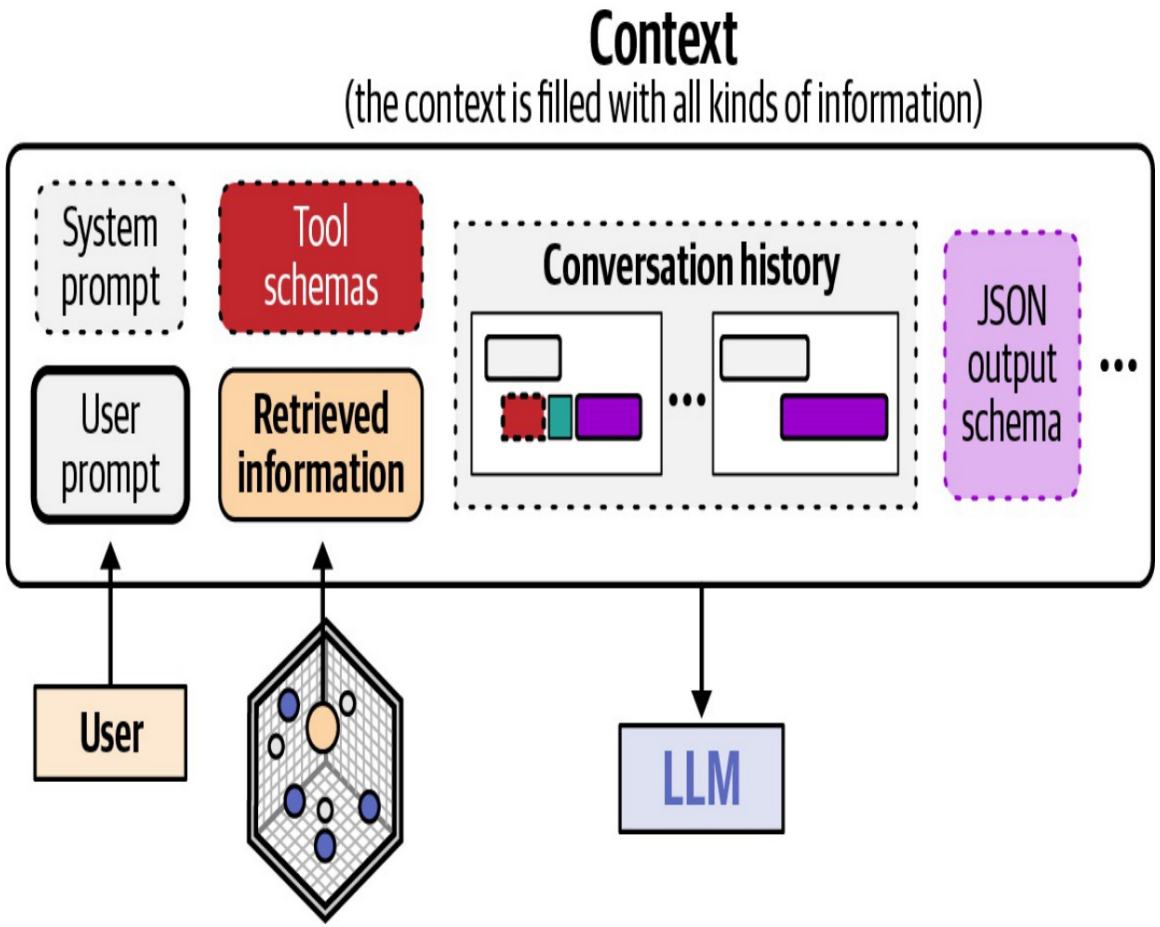
System prompt (procedural memory), conversation history (working memory), past experiences (episodic memory), and retrieved information (semantic memory). The figure also shows tool schemas and an output schema.

The LLM as a function

The model takes context tokens, processes them, and outputs tokens. The user's prompt is only a subset of the full context.

An open list

The list is not exhaustive. As models and agents develop, more kinds of information can be given to them.



Grootendorst & Alammar (2026). Fig. 4-27, PDF p. 224

Better output: optimize the input or the model

Two levers

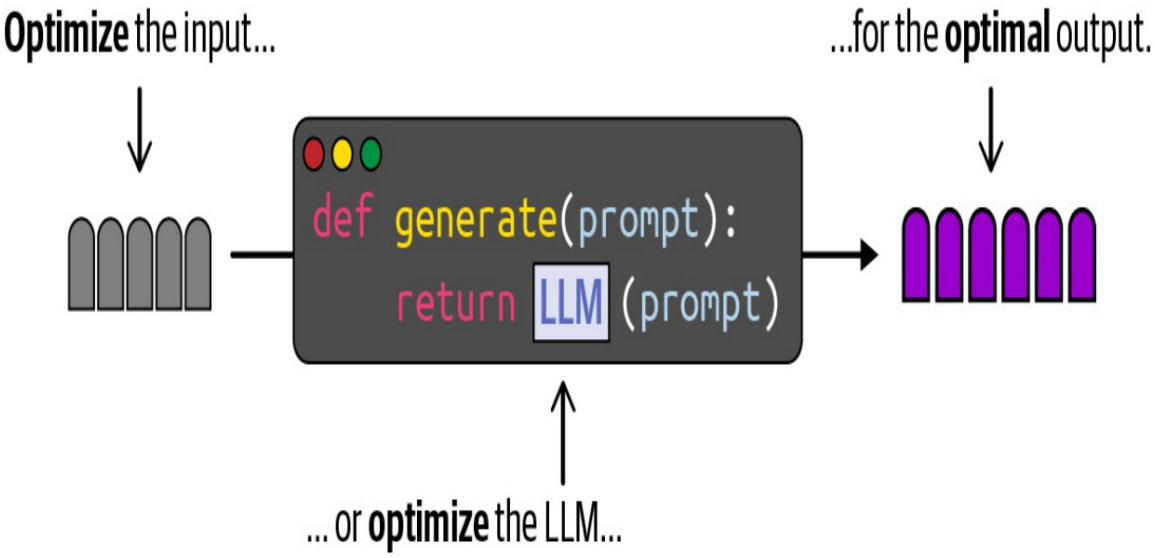
Output quality can be improved by training or fine-tuning the LLM, or by optimizing the tokens that go into it.

Context engineering

Optimizing input tokens so they produce the best possible output is context engineering. Formally, it is finding the context that maximizes output quality for a task.

The function view

Seen as a function, the context is the argument that developers can tune without retraining the model.



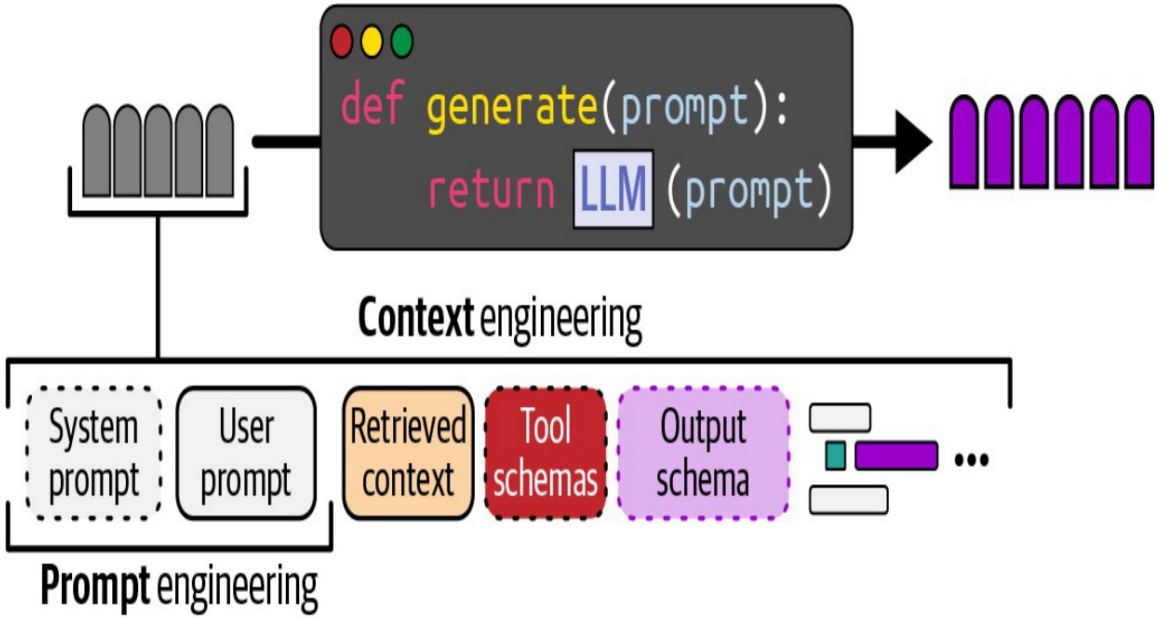
Grootendorst & Alammar (2026). Fig. 4-28, PDF p. 225

Prompt engineering is a part of context engineering

Prompt engineering
Optimizes the system and user prompts, giving the model the most relevant pieces of information.

Context engineering
Optimizes the entire context, which also includes retrieved context, tool schemas, and the output schema.

Different audiences
Prompt engineering is user-facing. Context engineering is a developer-oriented tool that also needs to communicate why the agent is executing particular tasks.



Grootendorst & Alammar (2026). Fig. 4-29, PDF p. 225

Bigger windows do not remove the need for engineering

Large windows

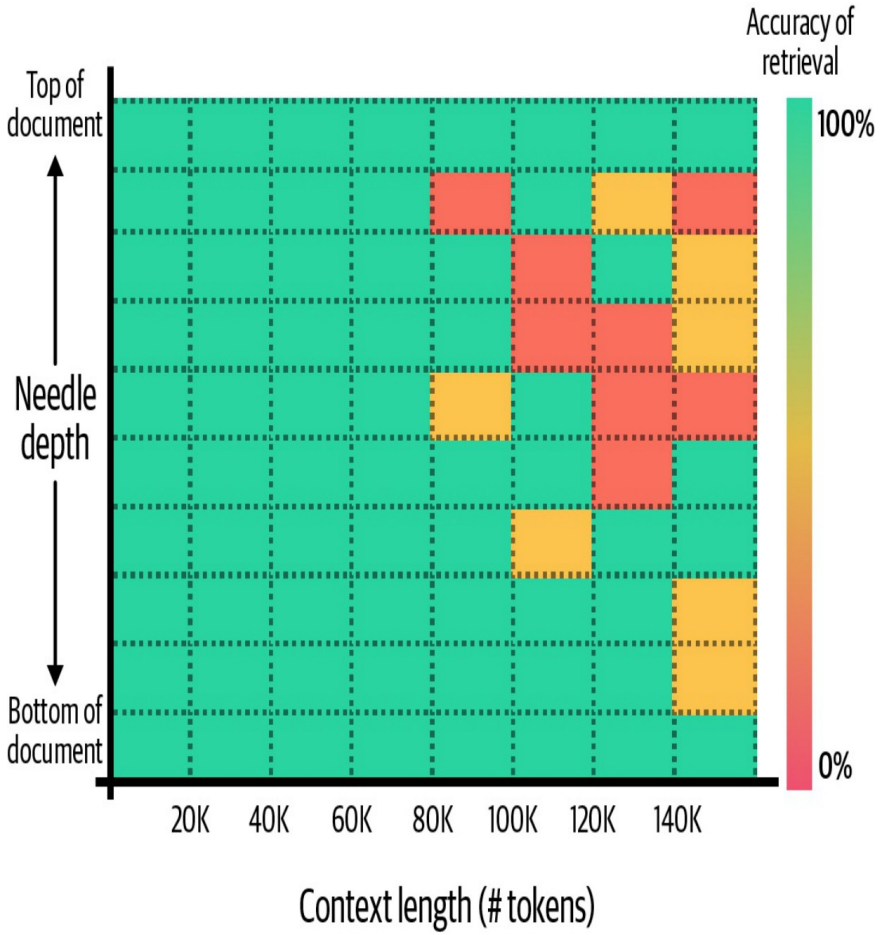
Gemini 1.5 reached a window of a million tokens in February 2024, which makes it tempting to fill the window and skip RAG.

Needle in a haystack

A random fact is placed at some depth in a long context and the model must retrieve it. The artificial heat map shows accuracy falling as the window fills.

Beyond retrieval

Finding a needle is only retrieval. RULER adds tasks such as multi-hop tracing and aggregation, and models that did well on needle tests dropped as length grew.



Grootendorst & Alammar (2026). Fig. 4-30, PDF p. 227

Why not simply fill the window?

Context rot

Several studies found that arbitrarily filling the window hurts performance, a pattern some call context rot. The quality of the information matters more than its quantity.

Attention

Even when the context is all high quality, models can struggle to sift through every piece. This may change as models improve.

Cost and latency

Every token must be processed, so long contexts increase latency. Memory requirements and cost also grow with context length.

Too little also fails

Too little context, or context in an inefficient form, leaves the model without a good understanding. The aim is just enough relevant information.

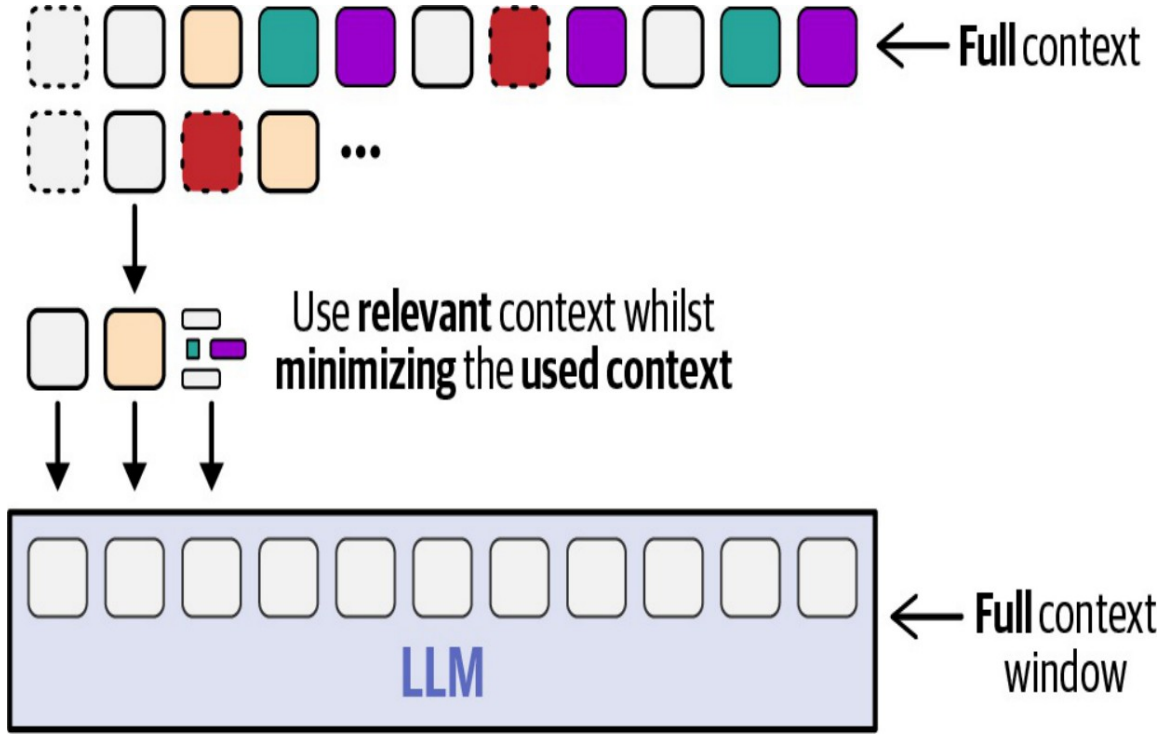
Grootendorst & Alammar (2026). PDF pp. 227–229

Right information, right place, right format

Strategic placement
From the full pool of possible context, choose only what is most relevant, and place it where the model can use it.

An architectural problem
Context engineering has many moving parts, including efficiently tracking, storing, and retrieving information that already exists and information that is created.

Reuse earlier ideas
MemoryBank adjusts the importance of memories and Search-o1 compresses retrieved context. Both are dynamic techniques that help as information flows grow more complex.



Grootendorst & Alammar (2026). Fig. 4-31, PDF p. 228

What to track and store

Agent behavior

Tool usage by the agent and its subagents, tool outputs and intermediate results, interactions between agents, reasoning steps, conversation history, and failures or successes.

User behavior

User intent, meaning explicit requests and goals, and user feedback such as edits, approvals, and rejections.

Knowledge sources

Snapshots of proprietary databases for reproducibility and auditing, external documents through RAG or APIs, and structured artifacts such as PLAN.md and REQUIREMENTS.md.

System-level

Configuration, such as model settings and available tools, and policies, such as guardrails and constraints. Privacy and safety constraints should also be included.

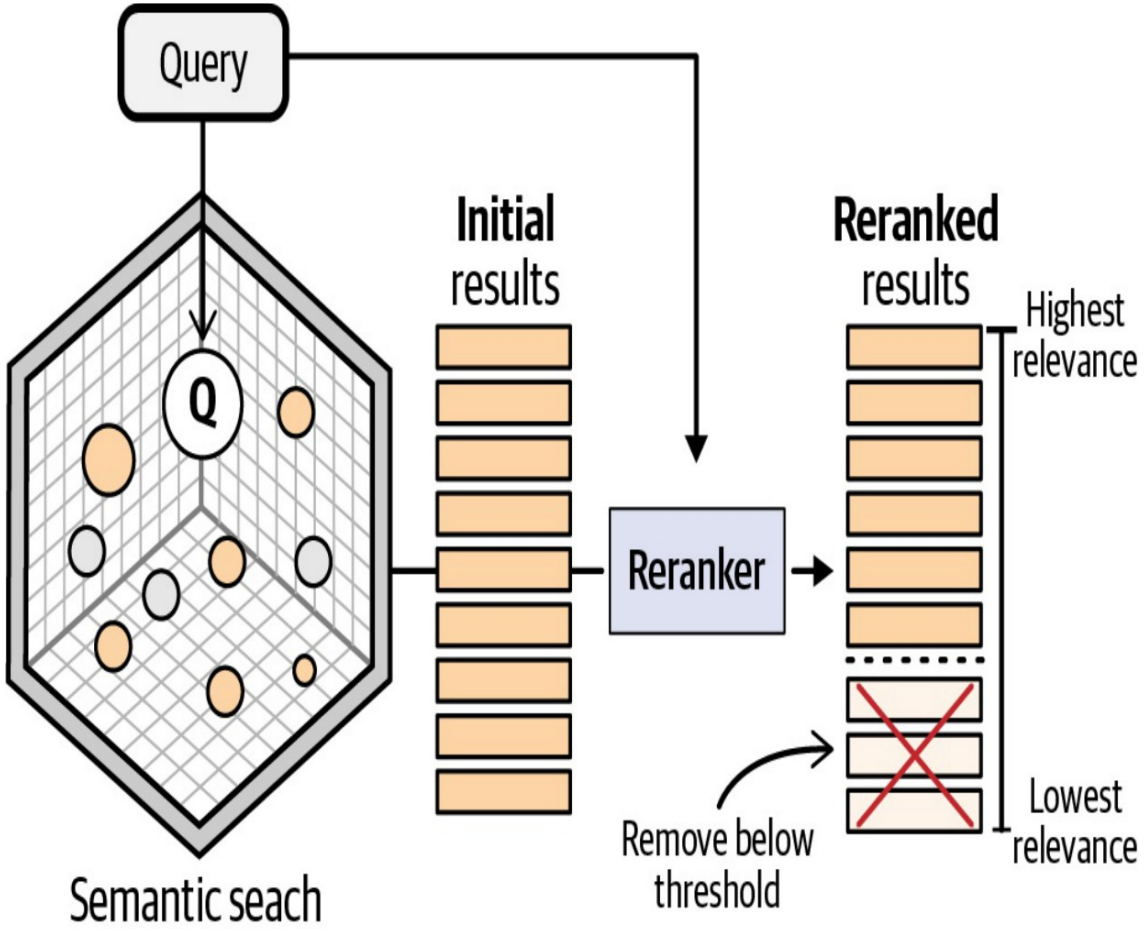
Grootendorst & Alammar (2026). PDF pp. 229–230

Reranking refines retrieved results

The problem
A search for the causes of climate change may return related documents about gases, industry, and deforestation that do not directly answer the question.

The reranker
A language model takes the query and the retrieved candidates and orders them by their relevance to the query and to each other.

A threshold
Results below a relevance threshold can be removed, so only the most relevant documents are kept. Reranking works on far fewer documents than the whole database.



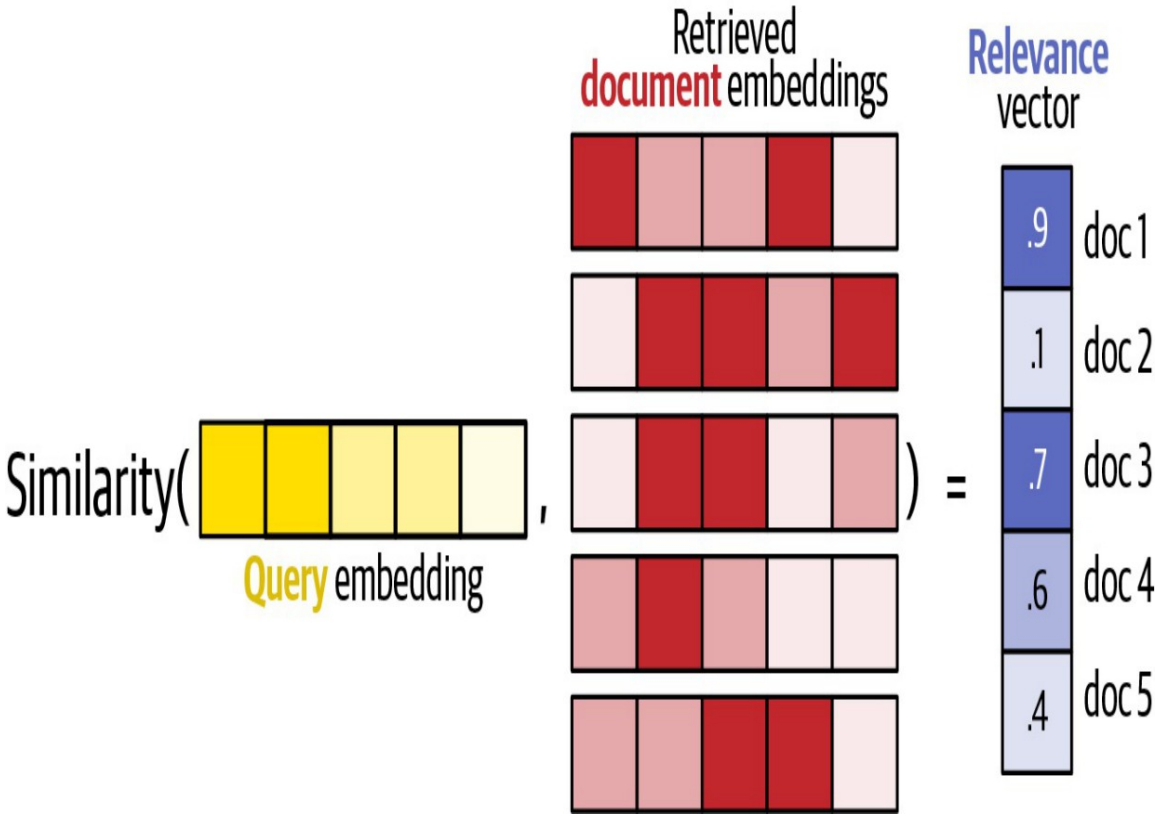
Grootendorst & Alammar (2026). Fig. 4-32, PDF p. 231

MMR step one: relevance to the query

Compression
Context engineering balances what goes in against how much goes in. Reducing redundancy is one compression strategy.

Redundancy
Even after reranking, the top five documents may say nearly the same thing, so together they add less than the sum of their parts.

Relevance vector
Maximal Marginal Relevance first computes the similarity between the query embedding and each document embedding, giving one relevance value per document.



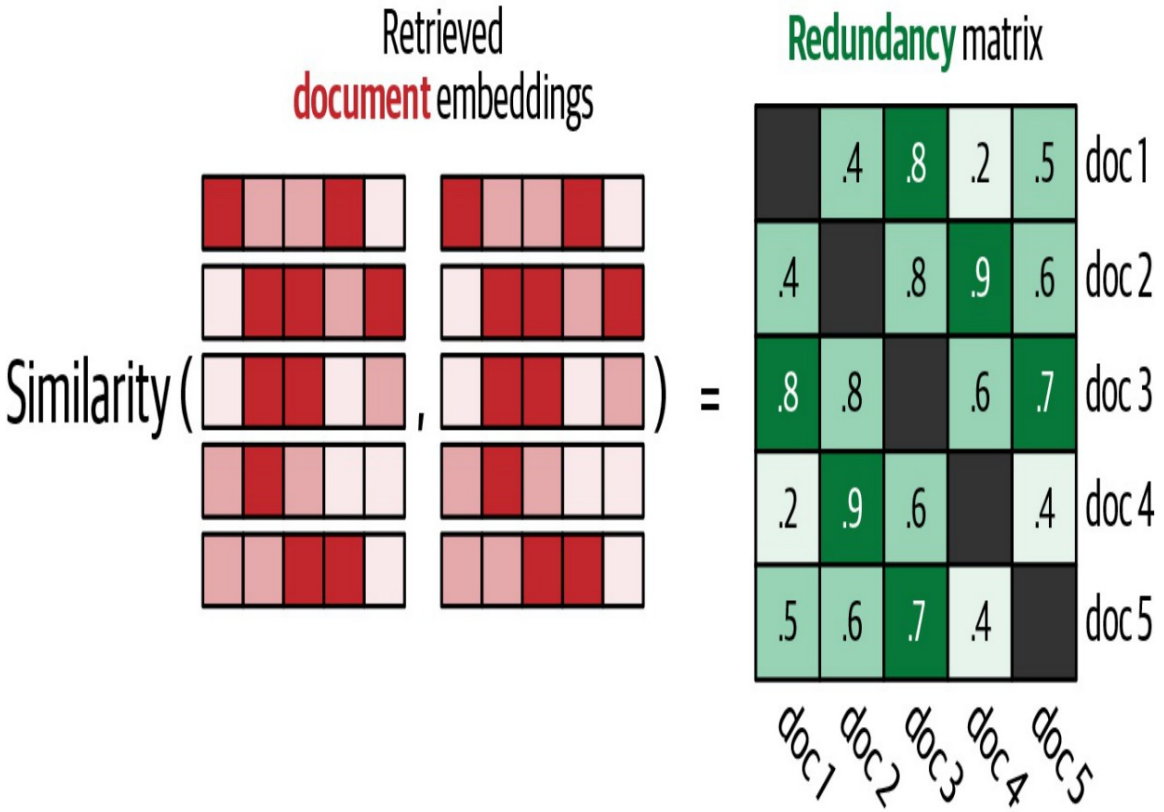
Grootendorst & Alammar (2026). Fig. 4-33, PDF p. 233

MMR step two: redundancy between documents

Redundancy matrix
Next, the similarity between every pair of retrieved documents is computed and stored in a matrix.

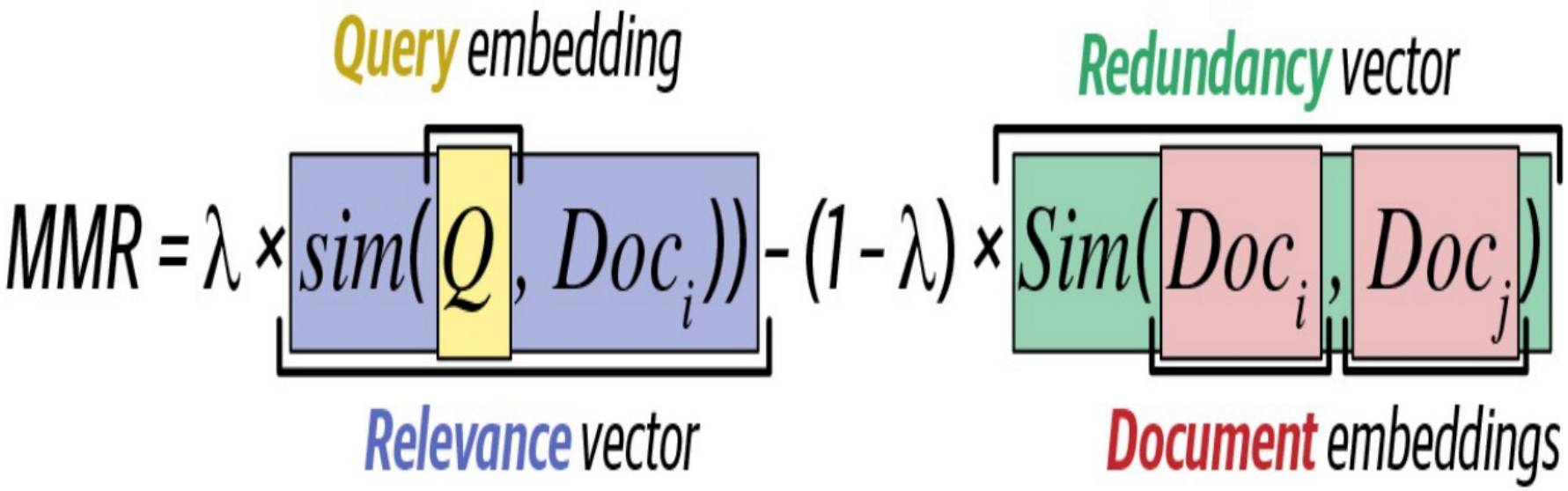
How it is used
A high value between two documents means that choosing one makes the other less useful. The matrix lets the method discard documents too similar to those already chosen.

The goal
Select documents that are similar enough to the query but dissimilar to the others already selected.



Grootendorst & Alammar (2026). Fig. 4-34, PDF p. 234

The MMR formula



Relevance term
Lambda multiplied by the similarity between the query and a document. This rewards documents that are relevant.

Redundancy term
One minus lambda, multiplied by the highest similarity between the document and any document already selected. This penalizes repetition.

Lambda
Lambda sets the balance between relevance and diversity. Choose the setting by testing it on the task.

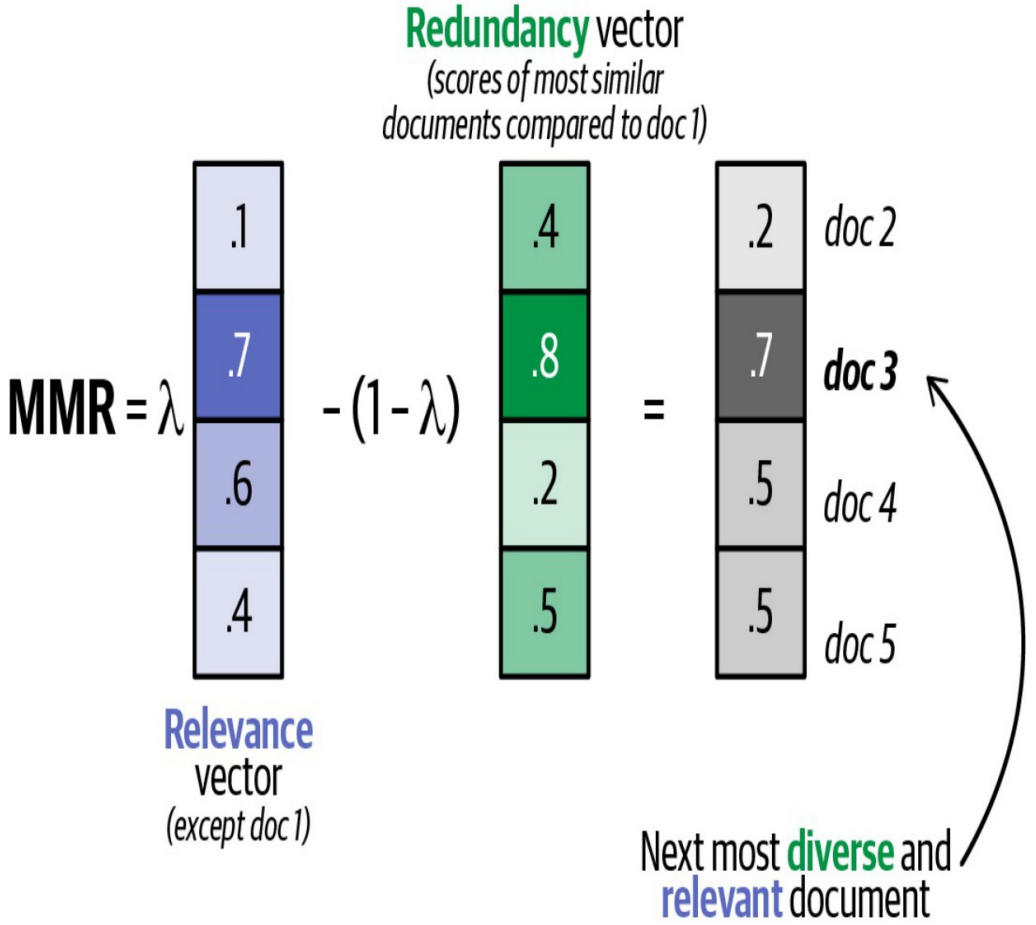
Grootendorst & Alammar (2026). Fig. 4-35, PDF p. 234

MMR step three: select the next document

Start
With nothing selected yet, choose the document with the highest relevance score, document 1 in the example.

Next
For each remaining document, multiply its relevance by lambda and subtract one minus lambda times its highest similarity to the selected set. Document 3 scores best.

Repeat
Continue, comparing against all selected documents, until the set is cut from five documents to three.

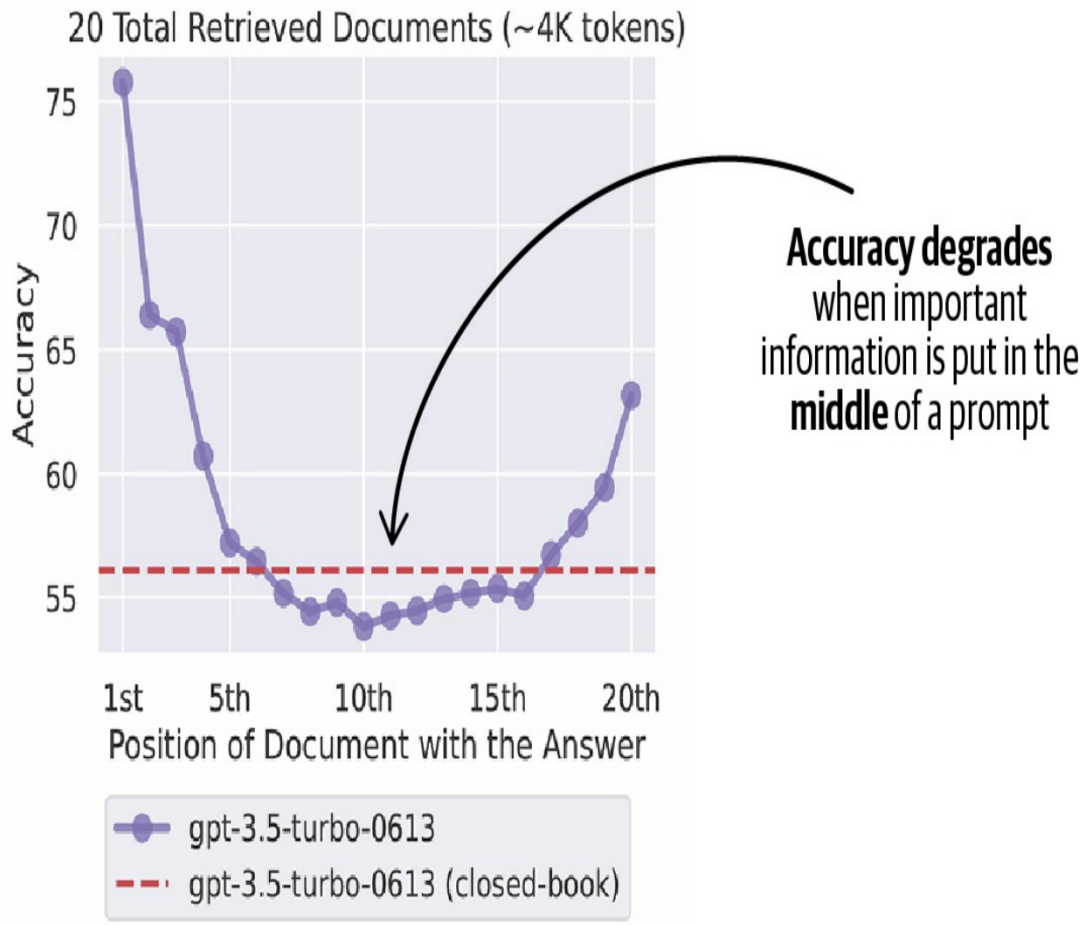


Position matters: lost in the middle

Beginning and end
Models tend to pay more attention to the beginning and end of a prompt and can lose information placed in the middle.

A human parallel
The serial-position effect describes similar human recall: the first items (primacy) and the last items (recency) are remembered best.

Practical meaning
The effect appears mainly with long contexts. Good context engineering helps by keeping the context focused and by ordering it carefully.



Grootendorst & Alammar (2026). Fig. 4-37, PDF p. 236

Context as the specification

A shift in mindset

The context given to an agent, including the query, PLAN.md, REQUIREMENTS.md, and the codebase, communicates intent to the agent and to the people working with it.

Keep the input

It is odd to keep only the output of a function. Tracking the context explains why an agent chose certain tools and actions, and it supports debugging and reproducibility.

Domain-specific

Health care needs very different context than law. There is no single framework, so developers consider domain sources such as patient data or research papers.

Many agents, many contexts

In multi-agent systems every agent's context must be managed. Small specialized agents handle smaller contexts, are easier to test, and leave the orchestrator more capacity.

Grootendorst & Alammar (2026). PDF pp. 229, 237

Learning check with worked answers

1. Why does an agent without Memory forget your name?
Each call contains only what the application sends. With no stored history, the earlier turn is not in the prompt.

2. When should an agent trim, and when summarize?
Trim when only recent turns matter and simplicity is valued. Summarize when older facts may matter and the extra model calls are acceptable.

3. How does agentic RAG differ from RAG?
RAG retrieves in a fixed step before generation. Agentic RAG lets the agent choose sources and decide how many retrievals to run.

4. Why not fill a million-token window?
Cost, latency, and context rot grow with length, and models can lose information in the middle. Select, compress, and order the context instead.

Core distinctions and source guide

Conceptual distinctions

Memory stores information, while context is what is placed in a particular call. Short-term memory serves the current task and long-term memory persists in external stores. RAG retrieves in a fixed step, while agentic RAG lets the agent decide. Prompt engineering tunes prompts, while context engineering tunes everything the model sees.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 4. Selected connections: Chapters 2, 3, 5, 6, 8, and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF